

Data Sparsity treatment for Context-Aware Recommender Systems



Gustavo Fleury Soares

Supervisor: PhD Elsa Negre

September 2020

Contents

1	Introduction	7
2	Recommender Systems	8
2.1	Recommendation Problem	8
2.2	Evaluation Methods	9
2.3	Context-Aware Recommender Systems	10
2.4	Data Sparsity	11
2.4.1	Data Imputation	11
2.4.2	Exploit more information	16
2.4.3	Dimension Reduction	17
2.4.4	Sparsity Measures	21
3	Experimental Protocol	22
3.1	InCarsMusic Dataset	22
3.2	CARSKit Library	22
3.3	Word Embeddings for Context Generalization	23
3.3.1	Similarity Measures	25
3.4	Data Imputation in less sparse Matrix Segments	26
4	Results and Discussion	28
4.1	Word Embeddings for Context Generalization	28
4.1.1	Conditions Generalization	29
4.1.2	Factors and Conditions Generalization	31
4.2	Data Imputation in less sparse Matrix Segments	32
4.2.1	Most Frequent Rating	34
4.2.2	DataWig	36
4.2.3	Factorization Machines	37
4.2.4	Discussion	39
5	Conclusion and Future Work	40

List of Tables

2.1	Confusion Matrix.	9
3.1	CARS Datasets Examples. Adapted from [Ilarri et al., 2018]	22
3.2	Loose Format	27
3.3	Compact Format	27
3.4	Binary Format	27
3.5	User-Item Ratings Matrix	27
4.1	Factor and Contextual Conditions	28
4.2	Example of Transformed InCarMusic Segment Dataset	32
4.3	Correlated Columns Example.	33
4.4	Most Frequent Imputation Example.	34
4.5	DataWig Imputation Example.	36

List of Figures

2.1	Logistic Function and Model example. Adapted from Harrell Jr [2015]	13
2.2	SVM visual example. Adapted from Han et al. [2011]	13
2.3	Random Forest Model example. Adapted from Han et al. [2011]	14
2.4	Perceptron inputs/output and multi-layer NN. Adapted from Nielsen [2015] and Han et al. [2011]	15
2.5	Neural Network Back-propagation example. Adapted from LeCun et al. [2015]	15
2.6	PCA visual example. Adapted from Powell and Lehe [2015]	17
2.7	SVD visual example. Adapted from Aggarwal [2015]	18
2.8	LDA decomposition. Adapted from Bergamaschi and Po [2014]	18
2.9	Data Sparsity treatment approaches	20
3.1	Weekdays Dendrogram	24
3.2	Dimensions Titles Vectors and Dendrogram	24
3.3	Similarity Comparison	26
4.1	Different results using distinct pretrained word embeddings model	29
4.2	Combination of Conditions Generalization - MAE	30
4.3	Combination of Conditions Generalization - RMSE	30
4.4	Factors Generalization - MAE	31
4.5	Factors Generalization - RMSE	31
4.6	Different results using distinct pretrained word embeddings model	33
4.7	MAE and RMSE Results for each group imputation	35
4.8	MAE and RMSE Results for Cumulative group imputation - Most Frequent	35
4.9	Overall Sparsity for each group imputation - Most Frequent	36
4.10	MAE and RMSE Results for each group imputation - DataWig	37
4.11	MAE and RMSE Results for Cumulative group imputation - DataWig	37
4.12	Impact of Number of Iterations in the FM result and computation time	38
4.13	MAE and RMSE Results for each group imputation - FM	38
4.14	MAE and RMSE Results for Cumulative group imputation - FM	39

List of Abbreviations

DATA MANIPULATION

<i>kNN</i>	<i>k</i> Nearest Neighbor
<i>ALS</i>	Alternating Least Squares
<i>BOCF</i>	Baseline Observation Carried Forward
<i>CAMF</i>	Context-Aware Matrix Factorization
<i>CNN</i>	Convolutional Neural Network
<i>CSLM</i>	Context Sparse Linear Method
<i>DL</i>	Deep Learning
<i>FFM</i>	Field-Aware Factorization Machines
<i>FM</i>	Factorization Machines
<i>LDA</i>	Latent-Dirichlet Allocation
<i>LOCF</i>	Last Observation Carried Forward
<i>LR</i>	Linear Regression
<i>LSA</i>	Latent Semantic Allocation
<i>LSTM</i>	Long Short-Term Memory
<i>MAR</i>	Missing At Random
<i>MCMC</i>	Markov Chain Monte Carlo
<i>MF</i>	Matrix Factorization
<i>MICE</i>	Multivariate Imputation by Chained Equations
<i>ML</i>	Machine Learning
<i>NB</i>	Naive Bayes
<i>NLP</i>	Natural Language Processing
<i>NN</i>	Neural Network
<i>PCA</i>	Principle Component Analysis
<i>RF</i>	Random Forest
<i>SGD</i>	Stochastic Gradient Descent
<i>SICE</i>	Sparse Inverse Covariance Estimator
<i>SVD</i>	Singular Value Decomposition
<i>SVM</i>	Support Vector Machines
<i>TF</i>	Tensor Factorization
<i>TF – IDF</i>	Term Frequency - Inverse Document Frequency

ERRORS MEASURES

<i>FN</i>	False Negative
<i>FP</i>	False Positive
<i>MAE</i>	Mean Absolute Error
<i>NDPM</i>	Normalized Distance-based Performance Measure
<i>RMSE</i>	Root Mean Square Error
<i>ROC Curve</i>	Receiver Operating Characteristic Curve
<i>TN</i>	True Negative
<i>TP</i>	True Positive

RECOMMENDER SYSTEMS

<i>CARS</i>	Context-Aware Recommender System
<i>CB</i>	Content-Based
<i>CF</i>	Collaborative Filtering
<i>KD</i>	Knowledge-Based
<i>OLAP</i>	Online Analytical Processing
<i>RS</i>	Recommender System
SPARSITY MEASURES	
<i>OS</i>	Overall Sparsity
<i>UIS</i>	User-Item Specific Sparsity
<i>USS</i>	User Specific Sparsity

Chapter 1

Introduction

Recommender System (RS) is a technology to suggest useful information for a specific user among many options available [Ricci et al., 2015]. As the amount of information is growing exponentially, identify the piece of data that may be of interest to the user is the challenge of these systems [Ricci et al., 2011].

The most common RS focuses on suggesting *Items* (e.g., books, news or products) for *Users*, working with these two dimensions, where users make evaluations (rates) for the items. Based on the similarity from the users or items, the RS could be a model to suggest the N items with the best score for an interest to a certain user. To improve the relevance of the recommendations it is interesting integrate contextual data [Adomavicius et al., 2005], like time, place and company of other person. This approach is known as Context-Aware Recommender Systems (CARS).

One of the primary challenges of RS is the few ratings available compared with the number of combinations of users and items. In real-world problems, the rating matrices tend to be very sparse, as users typically provide evaluation just for few items. And, it is very important obtain an effective ratings prediction from a small number of examples. This problem is known as data sparsity.

The integration of context adds new dimensions, however the availability of scores for all possible cases are still low, so it increases the problem of sparsity. The high data sparsity creates a challenge to find the similarities between Items and Users, limiting the quality of recommendations.

In this project, we review the basic RS methods and the existing researches to deal with the data sparsity in CARS environment and we propose two methods to treat the sparsity, with the purpose of obtain more useful recommendations. The first consists of generalizing the context using the word vectors associated to the factors names and context titles. The second uses different data imputation methods in less sparse matrix segment.

Chapter 2

Recommender Systems

2.1 Recommendation Problem

Recommender System (RS) is a technology which aims to suggest useful *Items* to *Users*. Adomavicius et al. [2005] describe this relation as a two-dimension function, assigning a rating score for the items not rated before, like the following utility function $R(u, i)$, where R means Ratings, u for users and i for items.

$$R(u, i) : Users \times Items \longmapsto Ratings \quad (2.1)$$

Typically, RSs are classified in Collaborative Filtering (CF), Content-Based (CB), Knowledge-Based (KD) and Hybrid Approaches [Ricci et al., 2015, Jannach et al., 2010, Aggarwal et al., 2016]. These approaches are normally designed to predict the missing values of the user-item matrix. The CB methods make recommendations based on the similarity of users or items content (e.g. text description, tags, images, user age range, user sex). The CF recommends items that other users with similar interests liked in the past, considering the previous user preferences (ratings, visualizations or choices). The KD, classified as *conversational system* because its highly interactive characteristic, consider the constraints or requirements passed by the user to make the filtering recommendation [Ricci et al., 2015]. The Hybrid RS is the combination of the other described methods, which try to reduce the impact of the limitations of each approach. As in this project we focus in sparsity problem of ratings matrix, we focus in CF method.

The CF methods may be based on the user, item or mix of both [Deshpande and Karypis, 2004]. The user-based CF finds a set of users with similar interests of a given user, and then the preferred items for these similar users are suggested. Nonetheless, the item-based CF make the predictions using the similarity between items. The quality of CF recommendation depends on the method used to determine the k most similar users or items and the considered importance for each item [Hwangbo and Kim, 2017]. The CF techniques normally are divided in *Memory Based* (use a sample or the entire rating database to generate the prediction) or *Model Based* (learn a model to make predictions, based on probabilistic approach using Data Mining, Machine Learning and Artificial Intelligence) [Colombo-Mendoza et al., 2015].

The utility function $R(u, i)$ usually estimates a *single-criterion* value. Like presented by Adomavicius et al. [2011] some works considered the use of just one *criterion* as limited, because the rating or a recommendation of an item for a particular user may depend on more than one utility-related aspect. For example, one user could consider an item of good material quality but with a bad functionality. So, the

use of *multi-criteria* ratings (e.g. distinct ratings for the quality of product, quality of user support, time of deliver) could help to improve the quality of RS.

2.2 Evaluation Methods

The evaluation of the quality and the value of the RS, in design phase, is implemented off-line and apply different approaches to compare the results. The main metrics of evaluation of the algorithm is the *accuracy* related, as Herlocker et al. [2004] mention, there are several like *Mean Absolute Error* (MAE), *Root Mean Square Error* (RMSE), *Accuracy*, *Precision*, *Recall*, *F1 Score*, *ROC Curves*, *Prediction-Rating Correlation*, *Normalized Distance-based Performance Measure*. However, there are other parameters to be considered beyond the accuracy, that could consider specific characteristics of evaluations of RS, like e.g. *Serendipity*, *Novelty* and *Confidence*. The aforementioned methods are explained below.

The equations of MAE (equation: 2.2) and RMSE (equation: 2.3) use the errors $y_j - x_j$, where y_j is the predicted value and x_j is the true value. And n is the total number of ratings in the dataset. The lower the MAE or RMSE, the more accurately is the ratings predict system. Compared to MAE, RMSE penalizes large errors. There are variations using normalized and average for MAE and RMSE.

$$MAE = \frac{1}{n} \sum_{j=1}^n |y_j - x_j| \quad (2.2)$$

$$RMSE = \sqrt{\frac{1}{n} \sum_{j=1}^n (y_j - x_j)^2} \quad (2.3)$$

Some of these measures are based on the confusion matrix, presented in Table 2.1 ,that indicate the number of predictions correct and incorrect. It is divided in True Positive (TP), True Negative (TN), False Positive (FP) and False Negative (FN).

Table 2.1: Confusion Matrix.
True Category

		Positive	Negative	Total
Predicted	Positive	TP	FP	$TP + FP$
	Negative	FN	TN	$FN + TN$
Total		$TP + FN$	$FP + TN$	N

Based on these notations, we have the following measures:

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN} \quad (2.4)$$

$$Precision = \frac{TP}{TP + FN} \quad (2.5)$$

$$Recall = \frac{TN}{TN + FN} \quad (2.6)$$

$$F1Score = 2 \times \frac{Precision \times Recall}{Precision + Recall} \quad (2.7)$$

The *Receiver Operating Characteristic* (ROC) Curve, is a technique used to compare different models rather than estimate their performance independently [Ricci et al., 2011]. It is the relation between true-positive rate (Recall) and the false-positive rate (probability of false alarm) (2.8).

$$False - PositeRate = \frac{FP}{FP + TN} \quad (2.8)$$

Normalized Distance-based Performance Measure (NDPM) is a metric designed for evaluating ranking systems. It measures the difference between the users rank and the estimated rank [Sidige et al., 2011]. NDPM is not applicable for evaluating unique predicted value.

The *Coverage* evaluates the whole RS, not just a ranked list, and it can be related to item space, user space or genre space coverage [Ricci et al., 2011, Silveira et al., 2019]. It refers to the extent of items (users or genre) that the RS is able to create predictions.

The *Novelty* is the ability of RS to consider the novel items in the recommendations. This metric is not trivial and would have to consider information out of the RS to verify if the user knows [Ricci et al., 2011]. The *Serendipity* considers the lucky finding or a satisfying surprise. It represents surprising recommendations [Ricci et al., 2011].

Among all the presented evaluation methods, the most used and accepted for comparison of the quality of predict engines for RS is MAE and RMSE. There are several articles that uses MAE and RMSE to compare and indicate the improvements [Jannach et al., 2010, Herlocker et al., 2004, Hofmann, 1999].

Off-line tests have the problem that provide no insight about the real user satisfaction, acceptance or experience with the system. Some *user-centered* evaluation is performed on-line, without the user being aware about the experiment, allowing to run several versions of RS in different groups of users. The comparison of results could be used to improve the system [Pu et al., 2011].

2.3 Context-Aware Recommender Systems

The afore mentioned recommendation methods presume that the future observations will follow the similar patterns previously collect in same environment. However, in real world, the environment is not stable, the users and items are continually changing, and the preferences are influenced by multiple distinct factors [Koren, 2009]. The *Context-Aware RS* (CARS) includes the contextual information, such as time of the day, place, and the company of other people [Adomavicius and Tuzhilin, 2011], for the purpose of obtain more relevant recommendations for the user in a given context [Zhou, 2019]. The contextual information could be e.g. as *temporal* (hour of day, weekday/weekend, season, events), *geographic position* (home/work, GPS coordinates), *environment* (temperature, weather, noise level, situation), *equipment* (parks, car parking, printer, garden, barbecue), *demographic data* (age, gender, nationality, financial income), *social company* (friends, children, wife/husband), *psychophysiological* (mood, tiredness level), and more.

Like discussed by Vahidi Ferdousi et al. [2017], the concept of *context-aware* is considered in several domains, such, among others, data mining, artificial intelligence, cognitive psychology, linguist. The widely accepted definition is from Abowd et al. [1999]:

“We define context as any information that can be used to characterize the situation of an entity, where an entity can be a person, place, or physical or computational object.”

In rating prediction, the problem of RS is estimate a utility function that predicts the rating from an user to an item. The objective is determine a utility function that minimize the errors from estimated and observed ratings. The other possibility for RS is indicate the *top-N recommendations* of items for an user. In CARS, instead use just 2 dimensions, like presented in eq. 2.1, we use the other context dimensions, like presented in eq. 2.9.

$$R(u, i) : Users \times Items \times \prod_{j=1}^n C_j \mapsto Ratings \quad (2.9)$$

where C_j represent the *contexts* and n the number of contexts.

2.4 Data Sparsity

Considering the CF approach, in typically real-world applications, the users interact (provide ratings, buy, click, see) for just few items from a very large catalog, generating a very sparse rating matrix. The RS challenge is to treat this sparse data in such way that provides good predictions [Jannach et al., 2010, Aggarwal et al., 2016]. And, in CARS, when are considered the context dimensions, the sparsity problem is heightened, with much less "ratings" available for which all possible condition.

These data sparsity causes some problems. In environments where new users or items are added to catalog, considering the already existing sparsity of data, generating recommendations for these new elements is difficult. This scenario is known as *cold-start* problem [Schein et al., 2002]. Likewise, the case where a user gives a rating for just few items, there is not enough data for RS to work accurately. This problem is known as *reduced coverage* problem [Su and Khoshgoftaar, 2009].

To deal with the sparsity problem in RS the main propositions are ignore, data imputation, exploit more information and dimension reduction. In Data Science environment, when a feature or an occurrence has few data and is considered not relevant, this part of data could be discarded [Van Buuren, 2018]. However, the structure of the RS ratings matrix is already very sparse and discard any information normally is not considered an option.

2.4.1 Data Imputation

Data Imputation consists in the substitution of the missing or inconsistent data field with estimated values. These estimated values are based on other available information.

Van Buuren [2018] presents some simple solutions for data imputation, that in some conditions could work:

- **Fixed Value:** in some cases, in continuous variables, missing values could be replaced by zero or a randomly selected value. Although it is an easy fix, it could impact in the proprieties of data and how to treat them.
- **Mean/Median/Mode Imputation:** a quick fix is replacing the missing data by the mean, median or *mode*, but it impacts some statistical parameters

and could disturb the relations between variables. Some *weighted average* or *median* could be applied to depending the type of data used.

- **Regression Imputation:** incorporates the information of the other variables to try imputing better values. With this method, the imputing predicted values can yield realistic results. One refined approach is the stochastic regression that adds some noise in predictions and still preserve the correlation between variables.

The methods of supervised Machine Learning (ML), for classification or regression, could be applied to assign value to missing data, by using the information of other features. In this way, a model is created using the existing values and applied to the missing values. Above, we explain some of the main ML methods:

- **Linear Regression (LR):** consists in find the 'best' line to fit two variables, like one variable can be used to predict the other. *Multiple* LR uses more than two variables to determine the model. For example (eq: 2.10) the modeled variable y is a linear function of another variable x . So, the variable y is estimated based on x value. The regression coefficients, w is the slope of the line and b is the y -intercept.

$$y = wx + b \quad (2.10)$$

To achieve linearity, some transformation functions could be applied, e.g., logarithmic ($y = w \log(x) + b$), exponential ($y = 10^{wx+b}$) and quadratic ($y = (wx + b)^2$).

- **Logistic Regression:** uses similar approach of Linear Regression using a Logit Function (equation 2.11) or Sigmoid Function [Harrell Jr, 2015]. The figure 2.1 exemplifies the function, where function P is the testing probability and the x the input variable. The right graph from figure 2.1 exemplifies application of Logit Function to input variable *age*. This method gives better results for problems nonlinear separable.

$$P(x) = \frac{1}{1 + e^x} \quad (2.11)$$

- **Naive Bayes (NB):** uses the Bayes Theorem, which calculates the probability of an event given the probability of other event occurred. The Bayes Theorem (equation 2.12), for calculate the probability of class " c " given a feature " d ". This method performs well in linear separable problems [Han et al., 2011].

$$P(c|d) = P(c) \frac{P(d|c)}{P(d)} \quad (2.12)$$

- **Support Vector Machines (SVM):** is applied to classify linear and nonlinear data. It transforms the data in high dimension using nonlinear mapping and searches the linear optimal hyper-plane in this new dimension. The figure 2.2 exemplifies the process, where the relation between the variable A_1 and A_2 is represented by the dots and the dots color represents the binary classification of classes. The objective of SVM is determine the division line with largest separation, like represented in right side of figure 2.2.

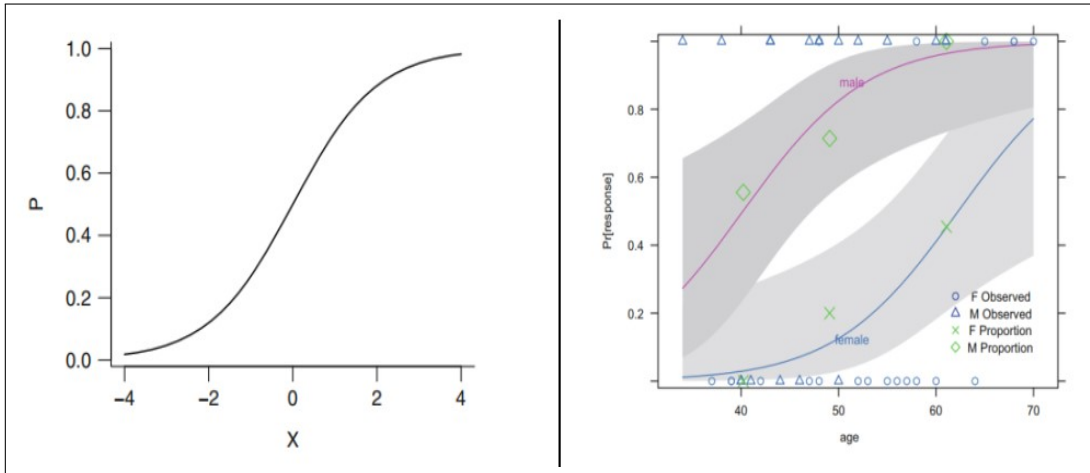


Figure 2.1: Logistic Function and Model example. Adapted from Harrell Jr [2015]

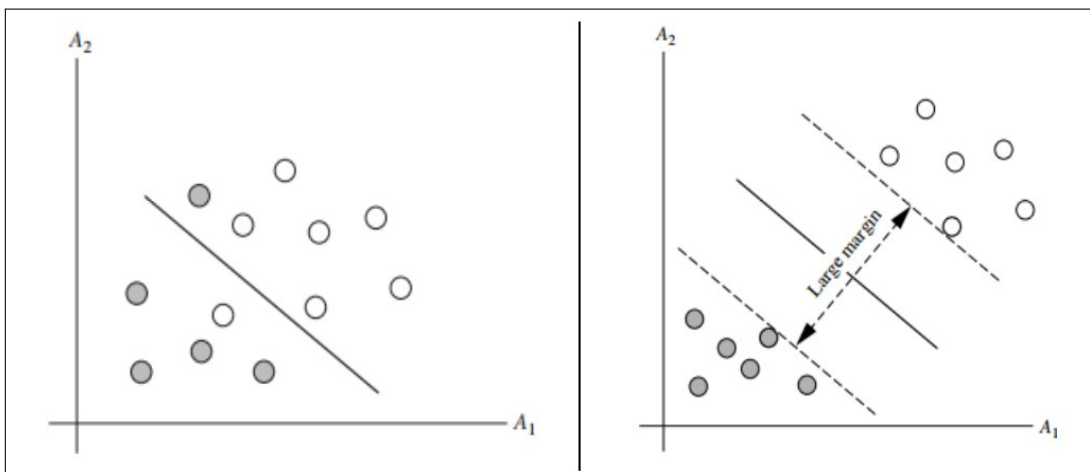


Figure 2.2: SVM visual example. Adapted from Han et al. [2011]

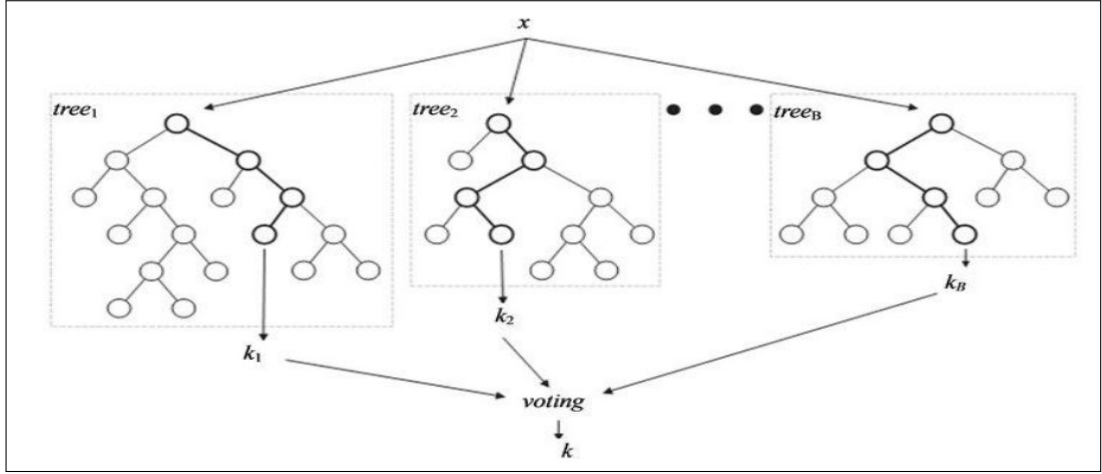


Figure 2.3: Random Forest Model example. Adapted from Han et al. [2011]

Some possible functions for transformation of new hyper-plane are Sigmoid Kernel (eq: 2.13), Gaussian Radial Kernel (eq: 2.14) and Polynomial Kernel (eq: 2.15).

$$K_{sig}(X_a, X_b) = \tanh(kX_a * X_b - \delta)^h \quad (2.13)$$

$$K_{gauss}(X_a, X_b) = e^{\frac{-|X_a - X_b|^2}{2\sigma^2}} \quad (2.14)$$

$$K_{poly}(X_a, X_b) = (X_a * X_b + 1)^h \quad (2.15)$$

where K represents the kernel functions, the X_a and X_b are the variables, and δ , h , σ are functions parameters.

- **Random Forest (RF)**: is an ensemble method for improve the performance of classification model. Ensemble methods combines a series of learned models with the aim to improve the results of classification [Han et al., 2011]. Others popular ensemble methods are Bagging, Boosting and Adaboost. The RF uses a “forest” of decisions trees generated using random selection of attributes (bootstrap). Each decision tree is considered, and the end take the majority of the votes for the classification. The figure 2.3 exemplifies the RF working method. There are created n trees, k_n possible classification, and finally the computing of voting for decide the class.
- **k Nearest Neighbor (k NN)**: is based on learning by analogy, by comparing k training tuples to similar neighbors. The distance between the neighbors is the Euclidean distance. k NN are widely to treat missing data and Batista and Monard [2003] indicate that it outperforms other methods.
- **Neural Network (NN)**: is a circuit of perceptron. A perceptron is an artificial neuron that receives some binary inputs $(x_1, x_2, \dots, x_n)$, apply some weights $(w_1, w_2, \dots, w_n)$ and produce single output [Nielsen, 2015, LeCun et al., 2015, Han et al., 2011]. Using the Back-propagation algorithm is possible to adjust the values of weights for the labels present in training dataset (supervised learning). Changing the weights values and adding more neurons and layers, it is possible to create NN for solve some problems. The figure 2.4 exemplifies the perceptron inputs $(x_1, x_2, \dots, x_n)$ and output (O_k) and a multi-layer NN, with the input, hidden and output layers. The figure 2.5

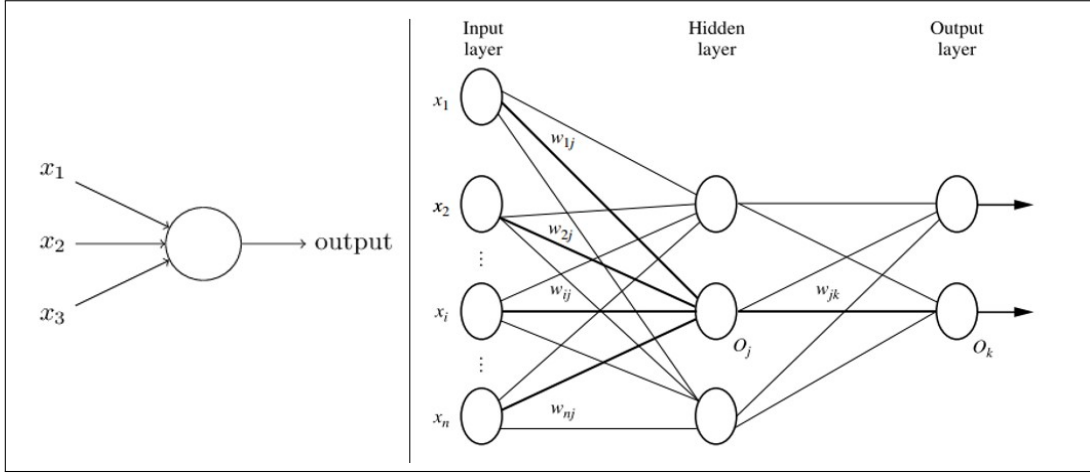


Figure 2.4: Perceptron inputs/output and multi-layer NN. Adapted from Nielsen [2015] and Han et al. [2011]

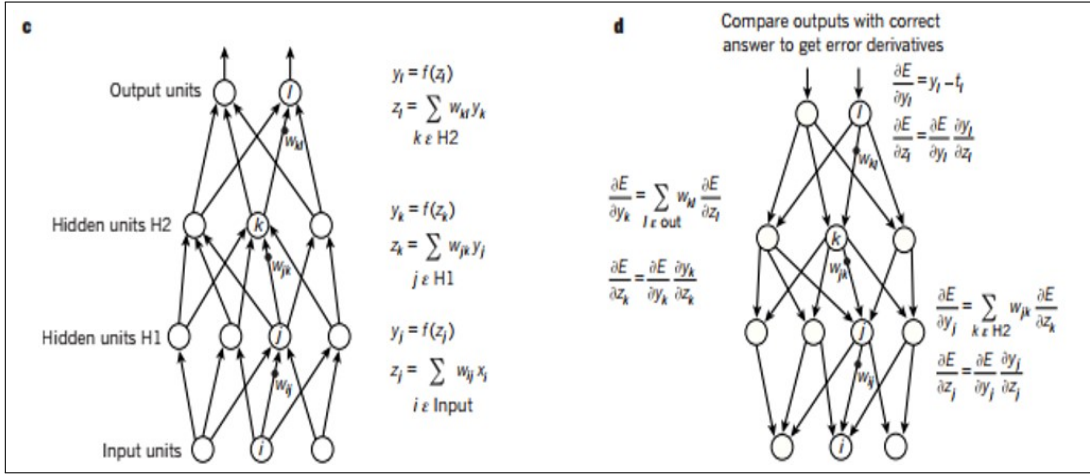


Figure 2.5: Neural Network Back-propagation example. Adapted from LeCun et al. [2015]

exemplifies the Back-propagation, where the outputs are used to adjust the weights of NN.

The Deep Learning (DL) is type of Machine Learning algorithms based in Artificial Neural Networks (NN), and its main idea is use several layers of NN in each one focus in a very specific simple abstract characteristic/task and train it in large scale with a big dataset. It simplifies the process of features engineering but requires a more data and computer processing [LeCun et al., 2015]. Normally classical ML could give better performance with small datasets. In another hand, DL improves more with more data.

For include product with few information (attributes or ratings) in the recommendations, Biessmann et al. [2018] propose a deep learning approach for imputation in unstructured text and categorical data, using a combination of numerical representation for categorical, embedding words and *Long Short Term Memory*(LSTM)/n-gram hashing (DataWIG). The result quality depends in the large amount of data and not very sparse data. Using a Hybrid RS, Kim et al. [2016] propose use if Convolutional Neural Networks (CNN) integrated into probabilistic matrix factorization (ConvMF) that captures contextual information of documents and improve the rating prediction quality. Zou et al. [2019] propose use a weight-Interaction layer

in a Neural Network (INN) to reduce the dimension of data sparsity and capture high-order latent patterns.

Some methods use the probabilistic characteristics of normal distribution, using certain assumptions about the missing data like the data is *Missing at Random* (MAR). Stadler and Buhlmann [2012] used a probability model based in the multivariate normal distribution and the dependencies among the variables, using the *Sparse Inverse Covariance Estimator* (SICE), to deal with missing values. *Multivariate Imputation by Chained Equations* (MICE) creates multiple imputations, resulting in less biased results [Azur et al., 2011]. In *chained equations*, the results of initial imputations are used as base to create new imputations.

There are several other methods of imputation, like *Last Observation Carried Forward* (LOCF), *Baseline Observation Carried Forward* (BOCF), *Indicator Method* [Miettinen, 1985], and all have specific type of data the is better applied.

The quality of ML prediction depends also on in the quality of data, it includes the completeness of data. So, most of the methods are impacted by the missing values. To verify the best imputation method for a given dataset, *multiple imputation* trying to complete datasets are proposed, with the analysis which one gives better results. The result could be created ensemble all the selected imputations methods, average of the values or by selection the most 'voted'. The confidence level in a particular imputed value could be represented by the variation across the different imputation methods applied [Van Buuren, 2018].

2.4.2 Exploit more information

Another common solution for address to missing data is try to find other related data sources or collect more data. The process of adding more information could be *explicit*, encouraging the users to make at least certain number of ratings, or *implicit*, collecting information from existing available data (e.g., log analysis, number of clicks, time seeing some product, information consulted) [Bobadilla et al., 2013].

Another possibility is combining outside information about the user in the system. The *cross-domain* RS import user preferences from other RS to the actual RS (e.g., the user's preference from bookstore could be used in music RS) [Hwangbo and Kim, 2017]. Using techniques to transfer knowledge from source domain to target domain, some researches indicate better results for RS [Cantador et al., 2015].

Niu et al. [2016] proposes to use *side information* from several online recommendation and review sites (e.g., movie or items comments, user profiles) and use this additional information to have a better recommendation. It is possible to use Natural Language Processing (NLP) methods to extract *tags* (information) from text fields, treat it using techniques like sentiment analysis or text summarization [Hu and Liu, 2004], and incorporate to existing RS.

The Semantic Web technologies is another source of information to deal with the problems of RS [Colombo-Mendoza et al., 2015]. Semantic Web allow computer systems work with data formats formerly readable by humans, and its main objective is better addressing the problem of information overload in Internet. The ontology or rule-based reasoning capabilities of Semantic Web are used to improve the quality of CF.

Another technique to improve the RS is mine the logs files for implicit information. For example, the number of clicks or the time that user stay looking in some item could give signals of user interest. This type of information could be collected by monitoring the user's behavior, such as songs heard, applications downloaded,

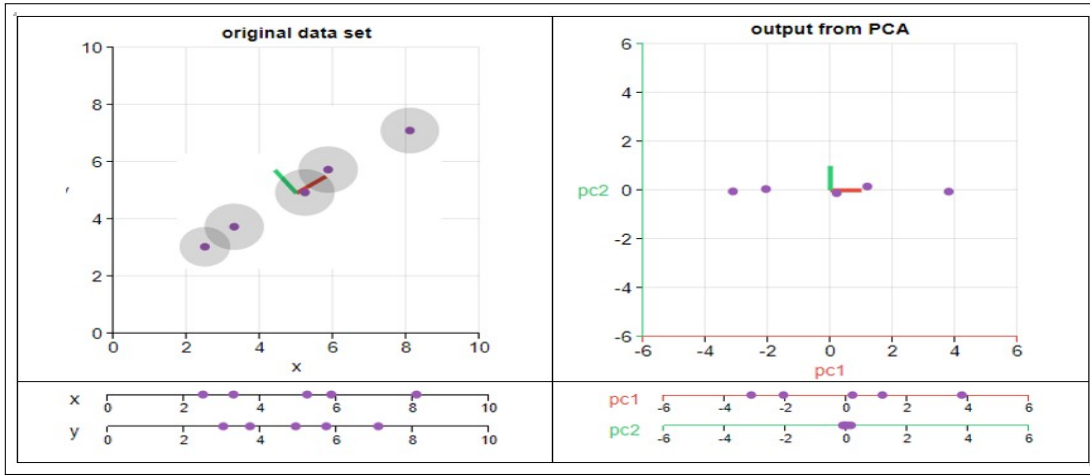


Figure 2.6: PCA visual example. Adapted from Powell and Lehe [2015]

web sites visited, ad clicks [Lee et al., 2010, Bobadilla et al., 2013]. These data are normally stored in logs files of the system and could be used like implicit data.

However, more generally that individual user’s context logs do not have enough data to create a model. Some researches propose use Latent-Dirichlet Allocation (LDA) or Latent Semantic Allocation (LSA) *Topic Modeling* for extract co-occurrences of user activities and associated contextual feature [Bergamaschi and Po, 2014, Krestel et al., 2009]. These approaches are explained in the next section.

2.4.3 Dimension Reduction

When there is a lack of data, another solution is to try to aggregate the various variables, generating a smaller quantity, but less sparse. In this respect, the *Dimension Reduction* consists in transform the original high-dimensional space into a lower-dimensionality, using much smaller volume, but maintains the characteristics of original data [Mining, 2006]. To treat the sparsity problem of RS, the dimension reduction is naturally applied.

The most relevant dimension reduction methods applied to RS are PCA [Nilashi et al., 2015, Vozalis and Margaritis, 2007], SVD [Bell et al., 2007, Brand, 2003], LSA [Bergamaschi and Po, 2014] and LDA [Krestel et al., 2009].

Principal Component Analysis (PCA) is a multivariate statistical method to structure, compress, interpret and visualize larger datasets [Nilashi et al., 2015]. It combines the essence of variables and create a smaller set of attributes [Han et al., 2011]. In RS, several researches apply PCA and some variances in distinct steps to provide the ranking recommendation [Nilashi et al., 2015, Vozalis and Margaritis, 2007]. PCA starts with the normalization of data, to prevent that large domains do not dominate the smaller domains. After it is calculated the orthogonal vectors, smaller than the original number of dimensions, that allow drop some attribute without lose much information. The figure 2.6 exemplifies how PCA works, where the 2 dimensions (left side) are transformed in just 1 dimension, with every points in same line (right side).

Singular Value Decomposition (SVD) is a matrix factorization method that generalizes the square matrix calculation of eigenvalues and eigenvectors for matrixes of size $d \times l$, where d is the dimensions and l the number of lines. The PCA and SVD are very similar, and SVD is more general. Like present in Aggarwal [2015], SVD provides the same basis vectors and data transformation as PCA for datasets

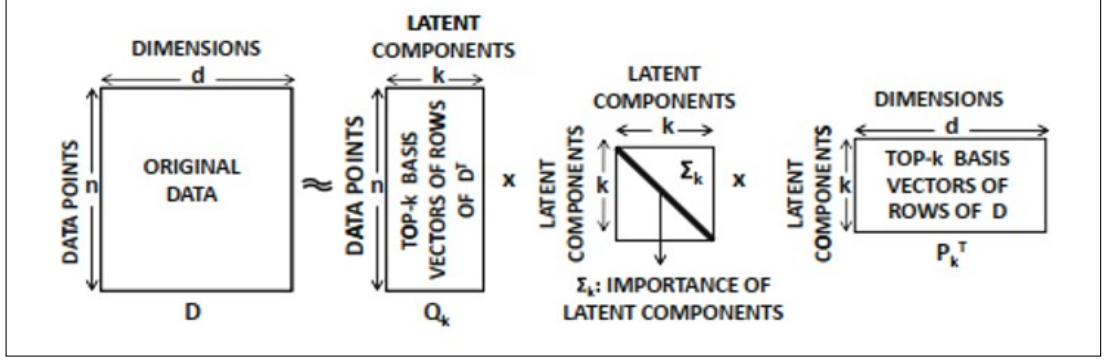


Figure 2.7: SVD visual example. Adapted from Aggarwal [2015]

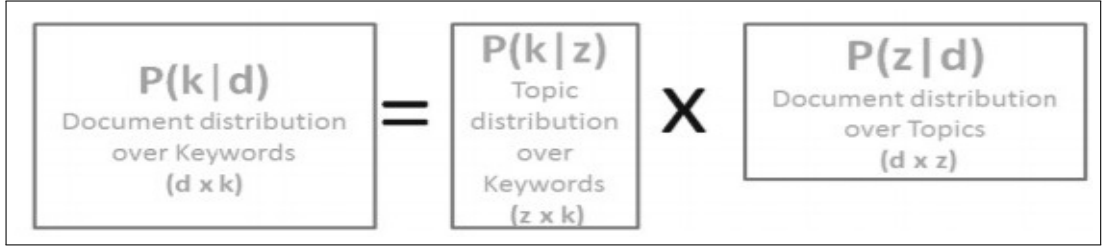


Figure 2.8: LDA decomposition. Adapted from Bergamaschi and Po [2014]

in which the mean of each attribute is zero. Like visually explained in the figure 2.7, the original data D is decomposed in 3 matrices that contains the importance of each component: Q_k , Σ_k and P_k , where k is the number of latent components.

Latent Semantic Analysis (LSA) extract the *contextual-usage* meaning of words considering some statistics metrics of a corpus of document [Bergamaschi and Po, 2014]. It consists in create a matrix $d \times k$, where d is the number of documents and k is the keywords (distinct words), using *Term Frequency - Inverse Document Frequency* (TF-IDF) or occurrence count [Ramos et al., 2003]. After is applied SVD method in this matrix to obtain the reduced dimensions.

Latent Dirichlet Allocation (LDA) is a probabilistic topic modeling which decompose the conditional probability of the keyword k given a document d , $p(k/d)$, in a probability of the term given the topic z , $p(t/z)$, and in a probability of the topic given a document, $p(z/d)$. Like LSA, LDA is designed to treat very sparse matrix, being very applicable to work with the content or context in RS [Bauman and Tuzhilin, 2014]. The figure 2.8 exemplifies the LDA decomposition, where the Document distribution over keywords matrix is decomposed in Topic distribution over keywords and Document distribution over topics matrices.

Factorization Machines (FM) are a model that combines *Support Vector Machines* (SVM) and matrix factorization [Rendle, 2010]. SVM is a linear classifier that fails to work with very sparse datasets. FM model presents good results with sparse data, being more general predictor compared with biased Matrix Factorization (MF) and SVD. In the MF the matrix is represented by equation 2.16. In the FM, are considered the linear relation and the context elements, like presented in equation 2.18. Observe that the first two elements of the FM characterizes a Linear Regression (LR) - equation 2.17 - and the last element the feature interactions.

$$D^{u \times j} = U^{u \times k} \times I^{k \times j} \quad (2.16)$$

$$\hat{Y}_{LR}(x) = w_0 + \sum_{j=1}^n w_j x_j \quad (2.17)$$

$$\hat{Y}_{FM}(x) = w_0 + \sum_{j=1}^n w_j x_j + \sum_{j=1}^n \sum_{z=i+1}^n (v_i \cdot v_z) x_i x_z \quad (2.18)$$

where $D^{u \times i}$ is the ratings matrix with dimension $u \times j$, decomposed in 2 matrices with k size in common. The w_0 is the global bias, w_j is the weight of j -th variable and the weight of the interaction of variables $x_j x_z$ is the factorization $(v_j \cdot v_z)$.

For CARS, there is an implementation of FM that considers the fields features, the *Field Aware Factorization Machines* (FFM). In FFM, each categorical value of each field (column) are considered a *feature*. Juan et al. [2016] indicate that FFM is better in very sparse dataset and using categorical features. Some approaches use Robust Optimization paradigm to reduce impact of perturbations [Punjabi and Bhatt, 2018].

To sparsity of ratings, Adomavicius and Tuzhilin [2001] propose to use the existing OLAP-like aggregation to generalize the existing information. In this way, the person belongs to a group, *graduate students*, or a day could be weekend (*Saturday* and *Sunday*) or workday (*Monday* to *Friday*). These types of generalization could be based, e.g., in explicit OLAP hierarchy (like word-days and weekends), text similarity, semantic relation [Colombo-Mendoza et al., 2015] or taxonomy. The text similarity uses the structure of the word to make the relations between of word. In another hand the semantic relation tries to use the meaning of the term to create the hierarchy. Some existing hierarchical and associative categories are normally as taxonomy, like websites classification based in a domain-specific ontology [Obeid et al., 2018].

The semantic relation of the words or terms consist and indicate a vector that characterize them, the embedding words. There are methods that determine in an unsupervised way a vector space representation of the words by analyzing their usage in a textual document's corpora [Mikolov et al., 2013]. The new approaches, with substantial better results, use pretrained results, treating large available documents corpora to characterize the words or terms using vectors [Pennington et al., 2014, Bojanowski et al., 2017]. In RS the word embeddings are frequently applied in analysis of the Content of Items (description and user evaluation of the items) [Musto et al., 2015].

In Stereotyping RS, the generalizations are made by some assumptions (e.g. mans like action movies and females like romantic movies), and the items are filtered using these profile user information [Lamche et al., 2014]. This approach is routinely applied in cold start problems.

To make the CARS less sensitive to sparse data, Phuong et al. [2019] propose to use segments of user-item graphs to augment the direct interactions. They propose a new graph-based similarity to select nearest neighbor recommendations, indicating that is possible to catch indirect associations among users and items. This approach is similar to pre-filtering with 'item splitting', that create fictitious items based on the contextual condition.

The figure 2.9 presents a main subject's chart about Data Sparsity treatment discussed in this section. This figure summarizes the main methods of treating sparsity, based on classical and recent research.

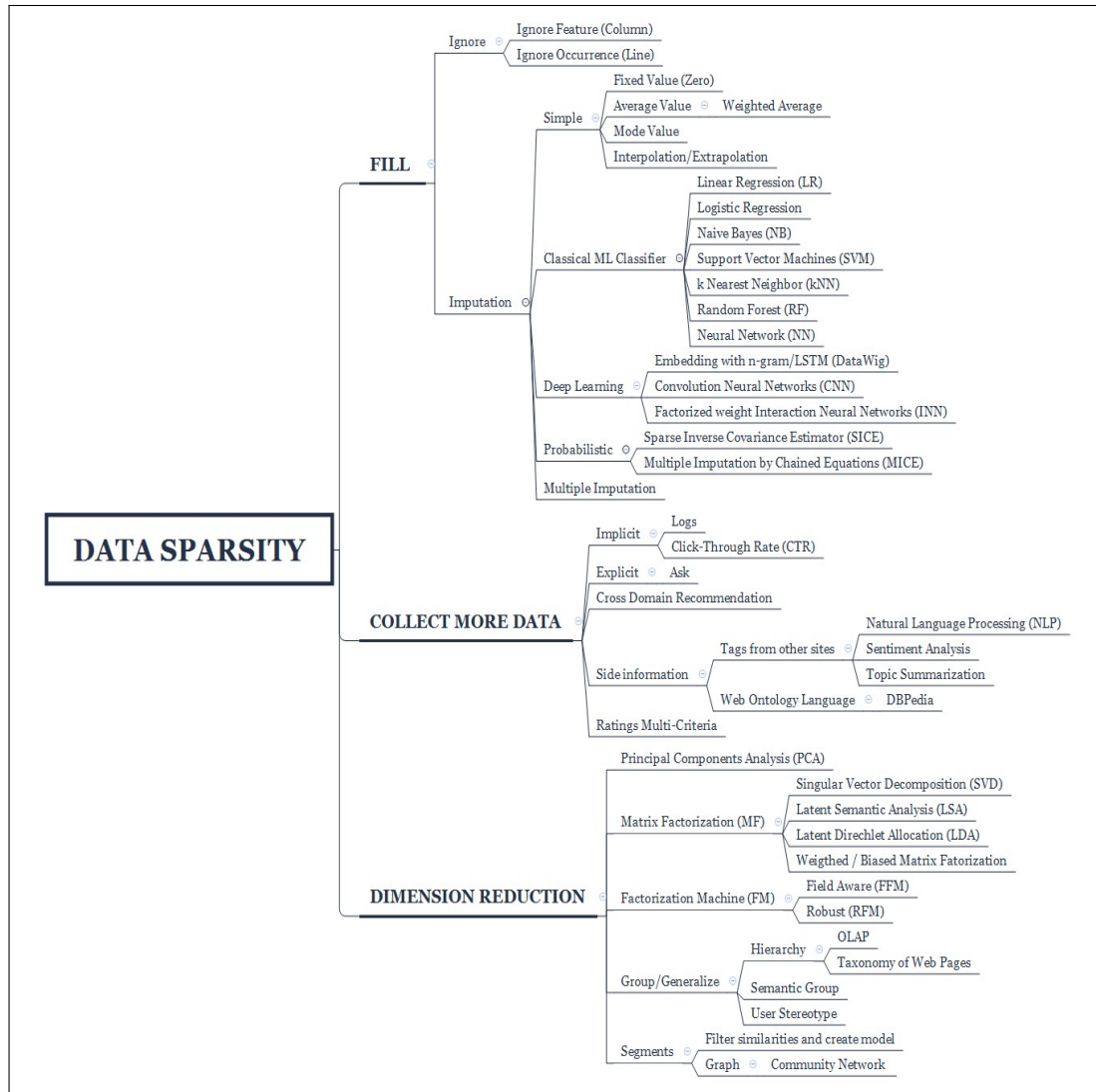


Figure 2.9: Data Sparsity treatment approaches

2.4.4 Sparsity Measures

To study the impact and compare the treatments of sparsity, it is necessary to use measures. There are several measures for the sparsity of a matrix. Anand and Bharadwaj [2011] exemplify some of these measures:

- Overall Sparsity (OS):

$$OS = 1 - \frac{|R|}{|U| * |I|} \quad (2.19)$$

where $|R|$ is the total number of ratings, $|U|$ and $|I|$ are number of distinct Users and Items, respectively.

- User Specific Sparsity (USS):

$$USS = 1 - \frac{|I_u|}{\max_{u \in U}(|I_u|)} \quad (2.20)$$

where $|I_u|$ is the number of items rated by the user u , and U is the user set.

- User-Item Specific Sparsity (UIS):

$$UIS = 1 - \frac{|V_{u,i}|}{G_u^i} \quad (2.21)$$

where $|V_{u,i}|$ is the number of local neighbors of user u , and G_u^i is the set of users who rated item i .

In the CARS, the Overall Sparsity considering the contexts could be measured by:

$$OS_{CARS} = 1 - \frac{|R|}{|U| * |I| * \prod_{t=1}^m |C_t|} \quad (2.22)$$

where $|R|$ is the total number of ratings, $|U|$ and $|I|$ are number of distinct Users and Items, respectively. And the $|C_t|$ is the cardinality of each context. Cardinality is the count of distinct values (the number of elements) of a set.

The addition of context dimensions increments the sparsity problem. For example, the InCarsMusic dataset [Baltrunas et al., 2011a], comparing the overall sparsity using $Users \times Items$ and with the $Users \times Items \times Contexts$:

$$OS_{RS}^{U \times I} = 1 - \frac{1914}{42 \times 139} = 67.2148\%$$

$$OS_{CARS}^{U \times I \times C_t} = 1 - \frac{3938}{42 \times 139 \times 9216} = 99.9927\%$$

For the InCarMusic dataset, in the CARS situation, the sparsity is incremented by the 8 factors (type context) with 16 different conditions (distinct values in a context) (table 4.1), given 9216 new possibilities ($2*3*4*2*3*4*4*4=9216$). These Observe that the purely $U \times I$ has less ratings available because some $U - I - R$ values repeat when we take out the context parameters.

The main metric used in researches about data sparsity is *Overall Sparsity* and it will be adopted to make comparisons in this work.

Like discussed, the Data Sparsity is very important problem in the RS, and there are different methods to treat it, normally divided in Data Imputation, Exploit more information or Dimension Reduction. To compare the impact of sparsity treatments is used the sparsity measures. In this project we tested combine Data Imputation and Dimension Reduction to treat the data sparsity.

Chapter 3

Experimental Protocol

In this section we will present the dataset studied, the CARS library used and the two main ideas tested for treat the data sparsity in CARS.

3.1 InCarsMusic Dataset

The common way to verify the quality of RS algorithms is use similar datasets and compare the results. For CARS there are few available datasets. Ilarri et al. [2018] describes the more used datasets for CARS, like present in table 3.1. As the main objective is study data sparsity, we decided to use InCarMusic dataset, because of its high sparsity.

The InCarsMusic Dataset was developed as APP to the listener rate the music in different contexts. There are 26 conditions (possible values) divided into 8 context factors (name of columns), like shown in table 4.1. The table 3.3 exemplifies the format the InCarsMusic dataset is originally distributed.

Dataset	# context attributes	% unknown ratings	References
InCarMusic	8	90.54%	Baltrunas et al. [2011a]
STS	14	89.37%	Braunhofer et al. [2014]
DePaulMovie	3	28.71%	Zheng et al. [2015]
Frappé	6	23.09%	Baltrunas et al. [2015]
LDOS-CoMoDa	12	4.00%	Košir et al. [2011]
TripAdvisor	1	0%	Zheng et al. [2012]

Table 3.1: CARS Datasets Examples. Adapted from [Ilarri et al., 2018]

3.2 CARSSKit Library

To verify if the proposed methods to treat the sparsity improved the quality of the recommendation we used an open-source Java-based context-aware recommendation engine: CARSSKit [Zheng et al., 2015]. It is specifically developed for CARS, contains the most common algorithms, with recent updates and widely is used by the RS research community [Haruna et al., 2017].

We used offline evaluation, measuring the recommendation’s accuracy based on ground truth test set. For test the accuracy we used the MAE an RMSE (equations 2.2 and 2.3, respectively).

The CARSKit provides a wide list of algorithms for traditional RS (UserKNN, BiasedMF, SVD++, etc.) and for CARS. The main CARS algorithms presented in CARSKit are listed below. Some variations in the algorithm’s behavior are possible, like presented by Baltrunas et al. [2011b] for *Matrix Factorization*:

- Context-Aware Matrix Factorization (CAMF):
 - Context (CAMF-C);
 - Context-Item (CAMF-CI);
 - Context-Item Category (CAMF-CC);
 - Context-User (CAMF-CU);
- Tensor Factorization (TF);
- Context Sparse Linear Method (CSLM);
- Factorization Machines (FM);

The library allows one to collect the metrics using all dataset with Cross Validation or using Training Dataset combine with Test Dataset. The k -fold Cross validation, also called rotation estimation, the dataset is randomly split into k mutually exclusive subsets of almost same size, where the model is trained and tested k times [Kohavi et al., 1995]. This method is normally applied to small datasets.

Zheng et al. [2014] verifies that CAMF-CU outperforms the other algorithms in some empirical studies. Based on this result, we will use of the algorithm CAMF-CU to verify the possible improvements in our data sparsity treatments.

We verified that the CARSKit results presents a small difference in the values, because of some random initialization in the developed algorithm. Like there is no option to seed, we decide to run the test some times and take the mean and standard deviation.

3.3 Word Embeddings for Context Generalization

The context generalization was described by Adomavicius and Tuzhilin [2001] to use the aggregation/OLAP capabilities. For example, the time dimension could be grouped in *weekdays* or *weekend*, and the products can be grouped according to the industrial classification. These hierarchies could be used to provide more complex recommendations, dealing with group of items.

However, not all dimensions have defined hierarchical group or relations between them. To address this problem, some researches use external aggregate ratings [Umyarov and Tuzhilin, 2007] or external side information [Niu et al., 2016], others take use of the internet taxonomy of websites [Colombo-Mendoza et al., 2015]. These approaches depend on external data, not always available.

The word embeddings are generally used to treat similarity in *Context-Aware* RS, transforming the items description in vectors [Ozsoy, 2016] or summarization [Hu and Liu, 2004]. Despite the existence of several approaches using word embeddings in RS, we did not find research that use them to create the context generalization.

Take advantage of the recent improvements in the methods of *Natural Language Processing* (NLP), we propose use of the pretrained model to create the relations between the contexts. These pretrained models use large databases and complex

methods [Pennington et al., 2014, Bojanowski et al., 2017, Devlin et al., 2018] to identify the relations of words or terms. In such manner, hierarchical relation could be created using latent knowledge present in the pretrained models.

To deal with the increased data sparsity of CARS, we propose use proximity of values of each dimension to group possible existing ratings and create generalized group context. Similar to hierarchies of days of week, we can group 'Monday to Friday' in *weekdays* and 'Saturday and Sunday' in *weekend*, we can use the proximity of word vectors to group the dimension values. The same approach is applied for the values of dimensions, grouping them using closer vectors. The figure 3.1 exemplifies it. In the left dendrogram the words 'thursday' and 'wednesday' are the closest to zero distance. After, 'saturday'- 'sunday' and 'friday'- 'monday' are grouped. And the 'tuesday' is the most distante vector. In the right dendrogram, we have the result of sum of vector by grouping the words 'sunday saturday'.

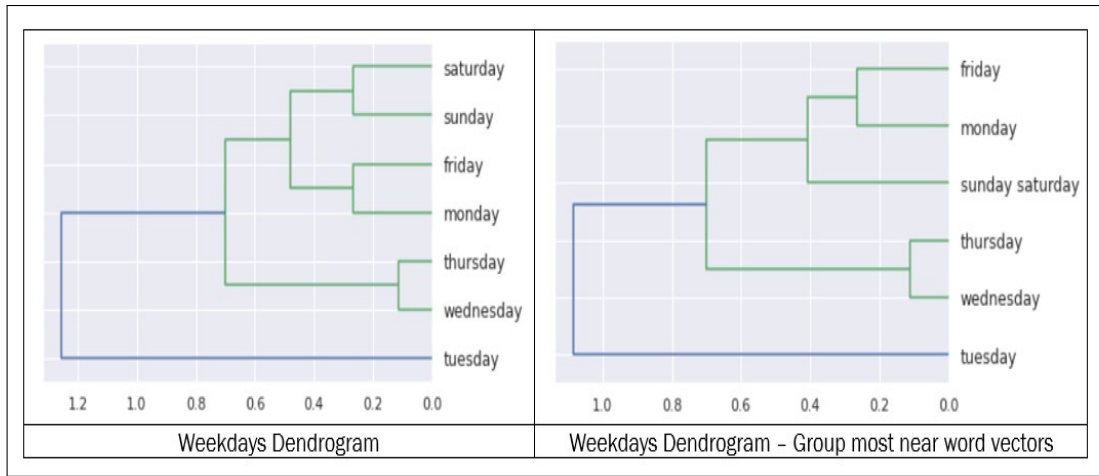


Figure 3.1: Weekdays Dendrogram

Similar approach to associate the dimensions (the columns), using their titles. For example, the titles of the dimensions are transformed in vectors and the closer vectors are associated, like presented in the figure 3.2. The left figure display the distribution of word vectors. And the right side is the dendrogram of context factors (columns titles).

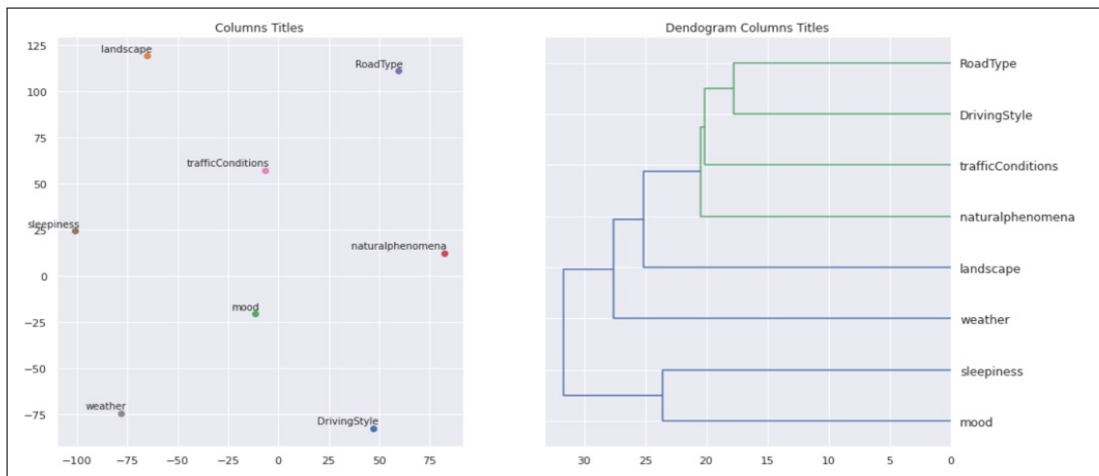


Figure 3.2: Dimensions Titles Vectors and Dendrogram

The use of pretrained word embeddings give more "meaning" in the proximity of

words, compared with a purely lexical similarity. The text similarity could be done, besides others, by exact match, bi/tri-gram character-based, phonetic proximity and Levenshtein distance [Gomaa et al., 2013]. This method search similarity in the structure of the string (word or phrase), do not associating with the semantic meaning. For example, the French words '*dessert*' and '*désert*' are very lexical similar, but with totally different meaning.

3.3.1 Similarity Measures

To calculate the agglomerative clustering for the word vectors is possible to use different distance methods. The distance methods are used to measure the similarity. Like discussed by Strehl et al. [2000], for meaningful clusters, the distance method and normalization could strongly affect in a positive or negative result. The main similarity measures are [Maimon and Rokach, 2005, Strehl et al., 2000]:

- Euclidean:

$$d_e(w, v) = \sqrt{\sum_{j=1}^n (w_j - v_j)^2} \quad (3.1)$$

- Cosine:

$$d_c(u, v) = 1 - \frac{w \cdot v}{||w||^2 ||v||^2} \quad (3.2)$$

- Correlation:

$$d_{cor}(w, v) = 1 - \frac{(w - \bar{w}) \cdot (v - \bar{v})}{||w - \bar{w}||^2 ||v - \bar{v}||^2} \quad (3.3)$$

- Extended Jaccard:

$$d_{jac}(w, v) = \frac{w^T \cdot v}{||w||^2 + ||v||^2} \quad (3.4)$$

Where n is number of dimensions, j is the index of dimension, $|| * ||^2$ is the 2-norm of its argument $*$, and $w \cdot v$ is the dot product of vectors w and v .

Some researches [Strehl et al., 2000, Ghosh and Strehl, 2006] indicate for cluster word embeddings vectors, the cosine and correlation presents better results. The Euclidean are considered not appropriate for high dimensional and sparse domains. The figure 3.3 presents an example of results using the different similarity measures.

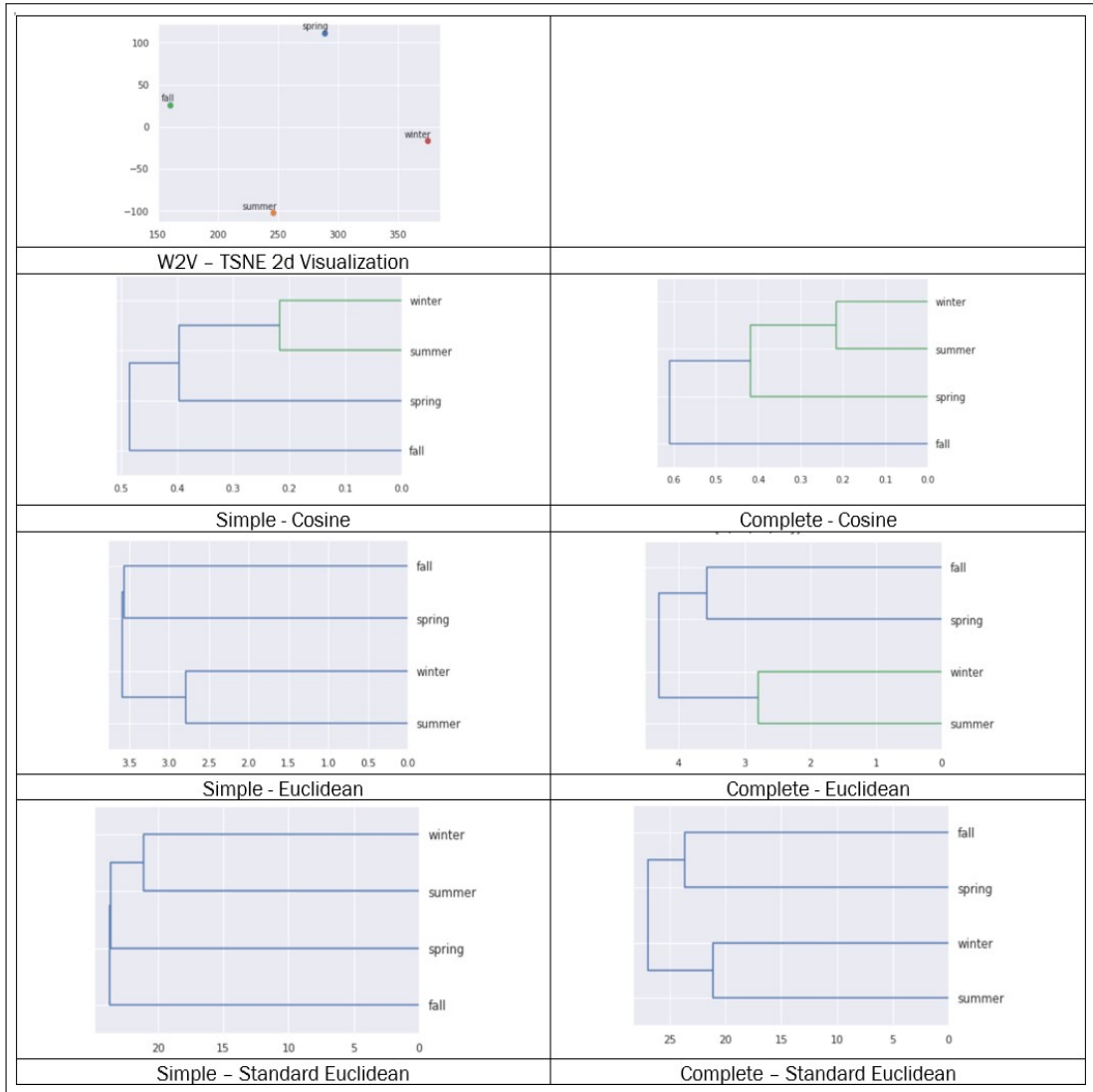


Figure 3.3: Similarity Comparison

3.4 Data Imputation in less sparse Matrix Segments

In RS, the data sparsity is very high, but the imputation methods work better when just few values are missing. All the system that process data are dependent of the quality of input data. To improve this quality, there are several imputation approaches. Although, most of these methods are very sensitive to sparse datasets [Rukat et al., 2019]. To deal with those limitation of imputation methods in sparse datasets of CARS, we propose to apply imputation in more dense segments of the dataset. The following topics detail the approaches.

In CARS, the ratings matrix are normally presented in loose, compact or binary format like exemplified in tables 3.2, 3.3 and 3.4, respectively [Zheng et al., 2015]. The 'NaN' represents the Null Values.

UserID	ItemID	Rating	Dimension	Condition
U1	I1	3	landscape	urban
U1	I1	4	landscape	mountains
U3	I2	5	weather	sunny

Table 3.2: Loose Format

UserID	ItemID	Rating	landscape	weather
U1	I1	3	urban	NaN
U1	I1	4	mountains	NaN
U3	I2	5	NaN	sunny

Table 3.3: Compact Format

UserID	ItemID	Rating	landscape: urban	landscape: mountains	weather: sunny
U1	I1	3	1	0	0
U1	I1	4	0	1	0
U3	I2	5	0	0	1

Table 3.4: Binary Format

To deal with the sparse data, we decided to reorganize the ratings matrix creating a new column grouping the 'User-Item' and create columns for each possible context. The table 3.5 exemplifies it.

UserID-ItemID	landscape: urban	landscape: mountains	weather: sunny
U1-I1	3	NaN	NaN
U1-I1	NaN	4	NaN
U3-I2	1	NaN	5

Table 3.5: User-Item Ratings Matrix

With this table format was possible to verify correlation between and create segments of correlated matrix to apply some imputation method to complete the missing data. After these steps, the new data was added to original rating matrix and the quality of recommendations was verified.

Phuong et al. [2019] used the user-item matrix with bipartite graph to calculate the similarity between users or items. There are different CF RS methods that combine User-Based and Item-Based to improve the quality of recommendations. However, no previous research was identified using the User-Item combination to address the Data Sparsity problem.

Chapter 4

Results and Discussion

The proposed methods to reduce the impact of data sparsity in RS was tested using the *InCarMusic* dataset [Baltrunas et al., 2011a] and the quality of recommendations using MAE and RMSE metrics using CAMF-CU method in CARSSKit [Zheng et al., 2015].

The *InCarMusic* is a CARS adapted dataset, with 8 contextual factors and with 26 contextual conditions (table 4.1), organized to verify the impact of music preferences of car passengers. Baltrunas et al. [2011a] verified that use Matrix Factorization (MF) considering the Context give smaller error (MAE) compared with purely MF.

Factor	Conditions
drivingStyle (D)	relaxed driving(r), sport driving(s)
road type (R)	city(c), highway(h), serpentine (s)
landscape (L)	cost line(c), country side(cs), mountains(m), urban(u)
sleepiness (S)	awake(a), sleepy(s)
trafficConditions (T)	free road(f), lots of cars(loc), traffic jam(t)
mood (M)	active(a), happy(h), lazy(l), sad(s)
weather (W)	cloudy(c), snowing(s), sunny(su), rainy(r)
naturalPhenomena (N)	day time(d), morning(m), night(n), afternoon(a)

Table 4.1: Factor and Contextual Conditions

4.1 Word Embeddings for Context Generalization

To apply pre-trained Word Embeddings, first we need to select between the available methods and after select the most adapted training source. Compared with most recent approaches, *FastText* present good results for small datasets and do not require a lot of computer resources [Ethayarajh, 2019]. If the dataset has long textual structures, *BERT*, *GPT-2* or *GPT-3* solutions are more appropriate.

We tested the InCarMusic dataset with *FastText* pre-trained models originated from distinct training sources and vectors size: Common Crawl, Wikipedia and DBPedia. The first two have 300 dimensions and are trained with a very big dataset. In other hand, DBPedia pre-trained model has just 10 dimensions. Because of the structured characteristic of DBPedia and the variables names and columns titles of CARS Dataset, the words proximity looks more human natural approximation. The figure 4.1 exemplifies the results using distinct models. In the left dendrogram, using Wikipedia model, the word 'saturday' is closer to 'friday'. In another hand,

in the right dendrogram, using DBPedia model, 'saturday' is closer to 'sunday'. Because the Context Factors (Titles) and Conditions (values) are composed for few words, a training dataset based in hierarchical data like DBPedia is more appropriated. DBPedia is project to extract structured information from Wikipedia [Auer et al., 2007], making the resulting information available on the Web for human and machine-consumption.

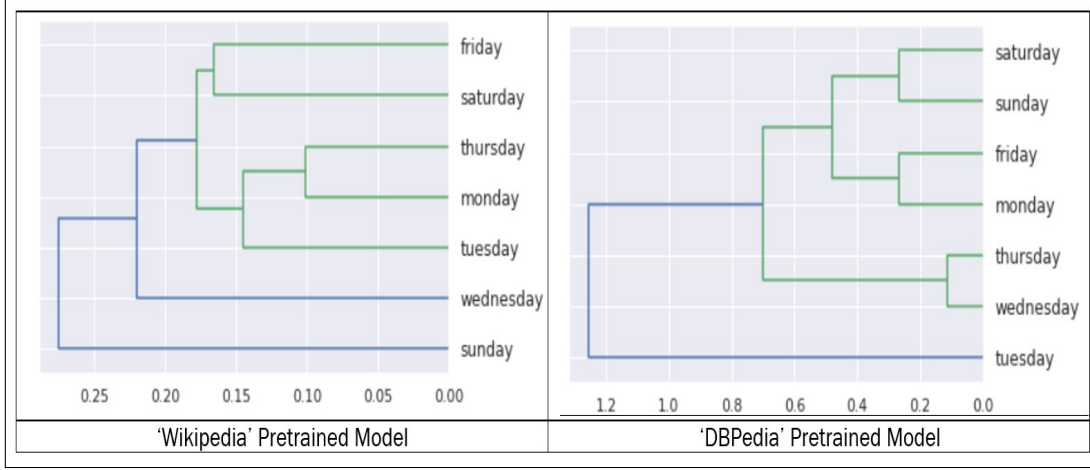


Figure 4.1: Different results using distinct pretrained word embeddings model

The tests done for Word Embeddings for Context Generalization follow the algorithms 1 and 2. The *Factors* are the titles of columns and *Conditions* are the values inside of columns. With this approach is possible to verify if the generalization has improved the recommendations. We collected MAE and RMSE using CARSSKit with the method CAMF-CU, testing all dataset, using crosstraining of 5 sub sets (k_fold).

Algorithm 1: Generalize Conditions

```

foreach Factor do
  while more than 2 Conditions do
    Generate Word Embeddings for Conditions;
    Group the 2 Closest Conditions;
    Collect MAE and RMSE with CARSSKit;
  end
end

```

Algorithm 2: Generalize Factors-Conditions

```

Combine Factors with Conditions names;
while more than 2 Factors do
  Group the 2 Closest Factors;
  Algorithm 1;
  Collect MAE and RMSE with CARSSKit;
end

```

4.1.1 Conditions Generalization

Doing all possible combinations of Factors with more than 2 conditions, we obtained the average values presented in figures 4.2 and 4.3. For identify which factor-conditions are generalized we used the first letter of each word, being factor in

The values of MAE and RMSE show little improvement in some combinations of Condition Generalization, where the average values are smaller than the Original test (orange dot line).



4.1.2 Factors and Conditions Generalization

After making all possible Conditions generalization, we applied similar approach to Factors, grouping the closest word vectors of Factors. The figures 4.4 and 4.5 present the results. In general, the title generalization did not give improvements.

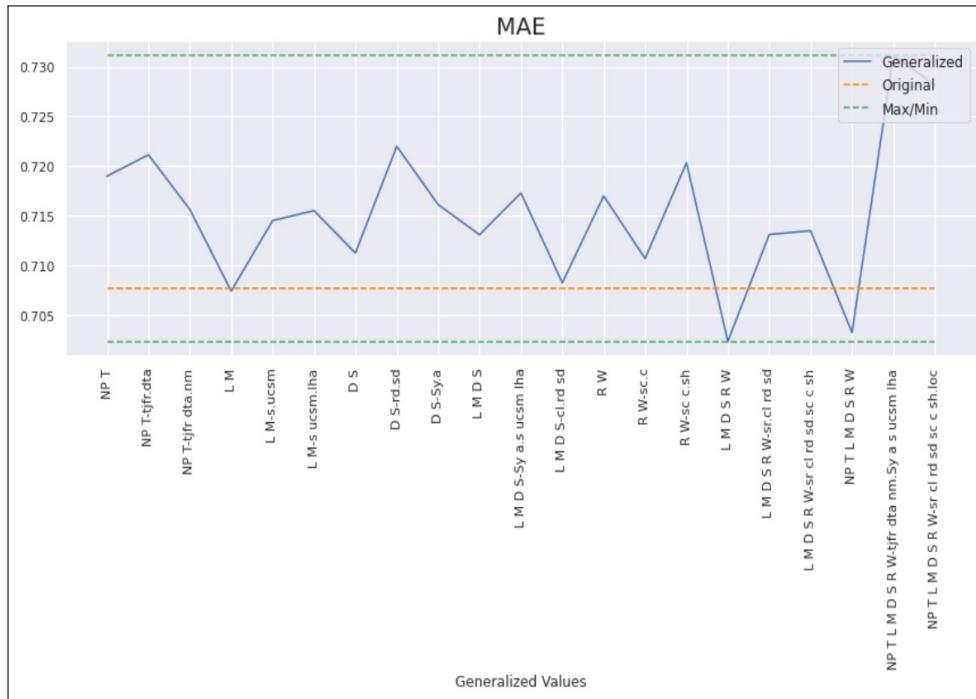


Figure 4.4: Factors Generalization - MAE

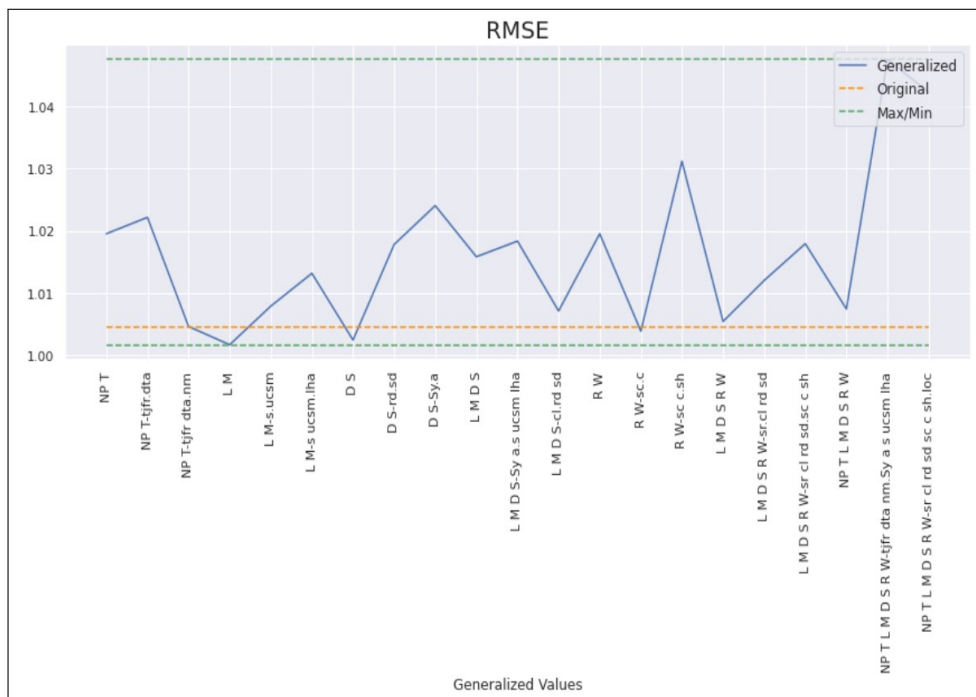


Figure 4.5: Factors Generalization - RMSE

4.2 Data Imputation in less sparse Matrix Segments

We make a transformation in the Rating Matrix to group the User-Item (UI) and create a column for each context condition. A transformed segment of the *InCarMusic* dataset is presented in the table 4.2. The column 'Main' represents the ratings when there is no context selected. Observe that in this data format, one line of User-Item (UI) register can have distinct ratings (one for each condition). In this dataset there is no register of combination of contexts, like '*DrivingStyle-relaxed driving*' and '*naturalphenomena -morning*'. In this type of transformation, the total of columns could be the combination of all context conditions. So, it is not a good format to store the data but an easy way to verify the correlation between the combination of contexts.

UI	main	DS - rd	DS - sd	NP - n	NP - m
1009-679	2.0	2.0	NaN	3.0	NaN
1009-752	NaN	2.0	NaN	2.0	NaN
1009-294	2.0	2.0	NaN	3.0	NaN
1019-754	1.0	1.0	NaN	1.0	NaN
1019-745	1.0	1.0	NaN	1.0	1.0
1026-294	3.0	3.0	NaN	3.0	NaN
1026-294	3.0	3.0	NaN	3.0	NaN

Table 4.2: Example of Transformed InCarMusic Segment Dataset

The columns '*main*', '*DrivingStyle-relaxed driving*' (DS-rd) and '*naturalphenomena -morning*' (NP-m) (table 4.2) are highly correlated. The figure 4.6 show the Pearson's Correlation of the columns in this new format. The values close to +1 or -1 indicates high degree of association between the two variables. As the coefficient goes towards zero, the variables relationship is weaker. For example, the variable 'RoadType-serpentine' is high correlated with '*trafficConditions-traffic jam*' (1.00), '*sleepiness-awake*' (0.97) and '*landscape-mountains*' (0.97).

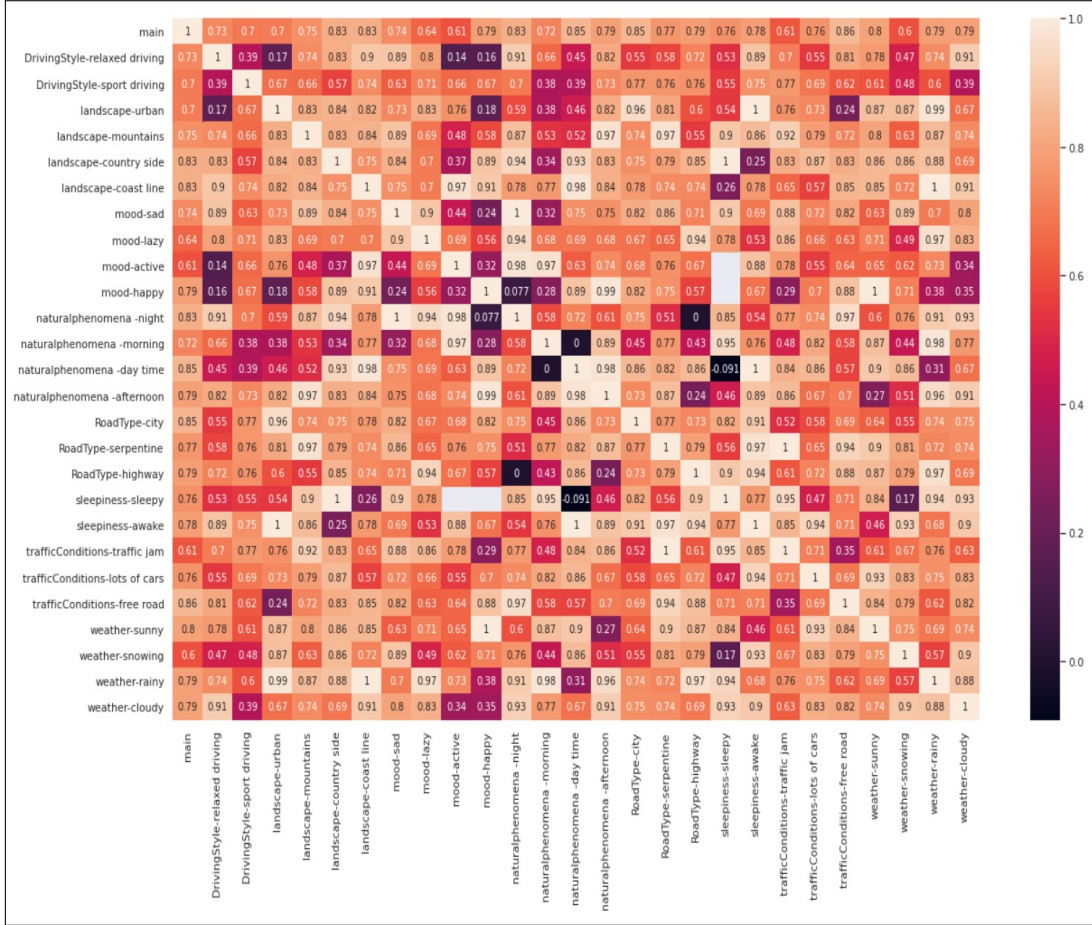


Figure 4.6: Different results using distinct pretrained word embeddings model

Using the correlation between the variables, we decided to apply data imputation in less sparse segments of matrix with high correlated columns that has ratings values. Using high correlated, we presumed that the imputation methods would have better results. The table 4.3 exemplifies a segment of high correlated columns (Pearson Coefficient bigger than 0.99). The new data are imputed in not rated context using different methods: most frequent rating value, DataWig (correlation and word embeddings of columns titles) and Factorization Machines (FM).

UI	trafficConditions- traffic jam	sleepiness- awake	landscape- mountains
1001-715	2.0	NaN	NaN
1001-279	NaN	2.0	NaN
1001-694	NaN	4.0	NaN
1001-251	NaN	2.0	NaN
1002-762	1.0	NaN	NaN

Table 4.3: Correlated Columns Example.

After the imputation, the new segment matrix is updated in the full matrix, and the quality of RS is tested using the CARSSKit. For verify if the imputation of each segment matrix could improve when tested together, we added a new step with the cumulative imputed matrices. In this case, the imputed values are appended in each step tested.

The tests done follow the steps:

Step 1: Split the dataset in *train* (80%) and *test* (20%)

Step 2: Verify Correlated Variables in *train* dataset.

Step 3: Make data imputation in the missing values (Most Frequent, DataWig, FM)

Step 4: Verify MAE and RMSE with imputed data and compare with original.

Step 5: Verify MAE and RMSE with cumulative imputed data and compare with original.

The following topics present the results of the test for the three different imputation methods. Like discussed before, considering that the CARSKit results has some variation, we decided to test 30 times each condition and use the mean value.

4.2.1 Most Frequent Rating

The simplest method tested, it consists to copy the most frequent rating for each UI line in high correlated matrix segment. For example, in table 4.3 the missing values (*NaN*) are replaced by the most frequent rating in UI occurrences. The table 4.4 exemplifies this method. The bold values are the original and the italic values are the imputed ratings.

UI	trafficConditions- traffic jam	sleepiness- awake	landscape- mountains
1001-715	2.0	<i>2.0</i>	<i>2.0</i>
1001-279	<i>2.0</i>	2.0	<i>2.0</i>
1001-694	<i>4.0</i>	4.0	<i>4.0</i>
1001-251	<i>2.0</i>	2.0	<i>2.0</i>
1002-762	1.0	<i>1.0</i>	<i>1.0</i>

Table 4.4: Most Frequent Imputation Example.

To verify in which group data imputation improve the quality of recommendations, we tested for each group of correlated columns. The figure 4.7 presents the MAE and RMSE results, with each group, where the imputation for each group of correlated columns are tested. And cumulative imputation (figure 4.8), where the imputation groups are concatenated in the same matrix and tested together. This example considered the high correlated columns those with correlation coefficient is equal or bigger than 0.99 (correlation threshold). For these training dataset (*numpy* library random seed = 2), there are 25 correlations groups with correlation coefficient bigger than threshold.

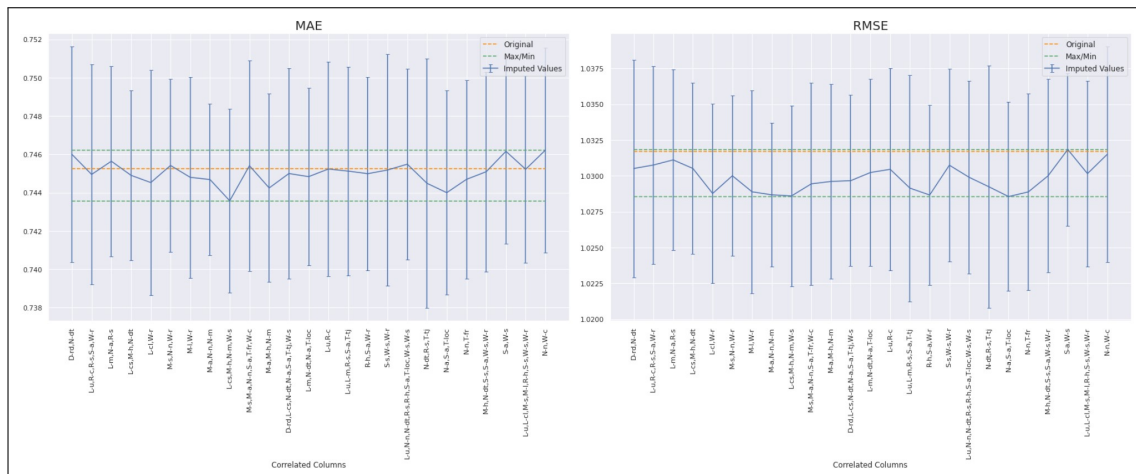


Figure 4.7: MAE and RMSE Results for each group imputation

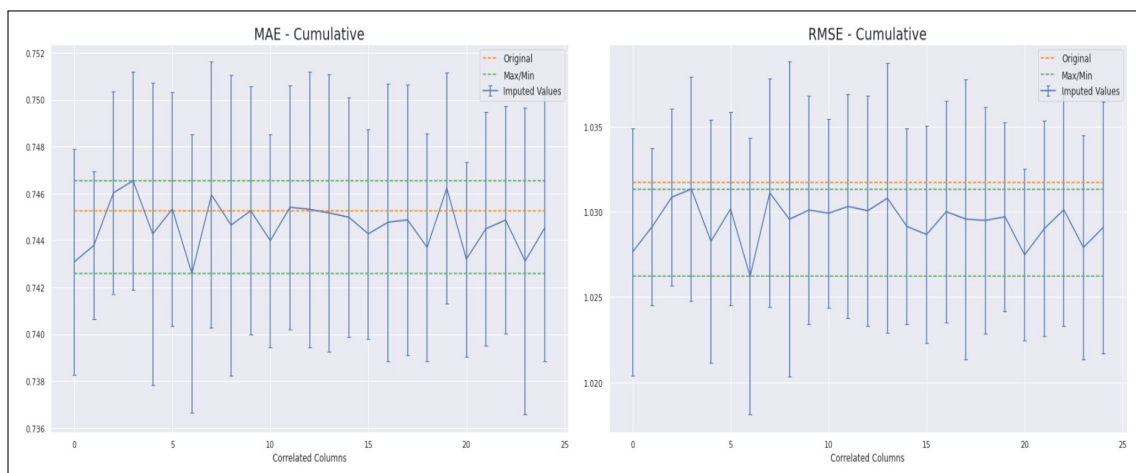


Figure 4.8: MAE and RMSE Results for Cumulative group imputation - Most Frequent

With the imputation the Overall Sparsity for each group are decreased, like presented in figure 4.9. It is possible to verify that the less sparse data does not mean direct better recommendation, but still have a correlation coefficient of 0.76. Like the imputation methods works in a very small segment of the Rating Matrix, the reduction of sparsity is small too.

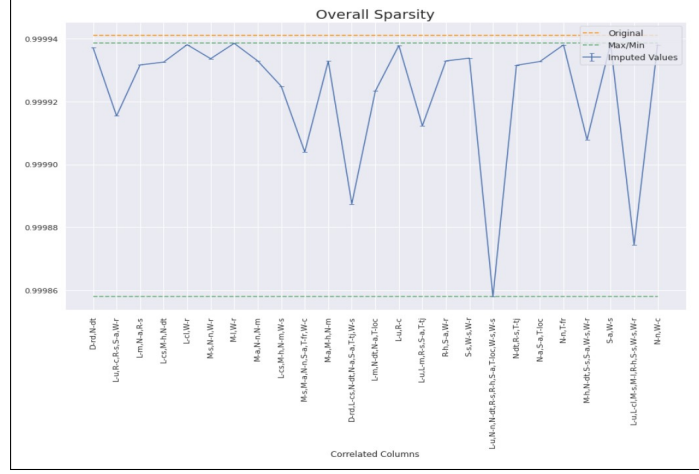


Figure 4.9: Overall Sparsity for each group imputation - Most Frequent

Like the CARSKit give results with little differences (because of bootstrap characteristic), sampling is necessary to statistically guarantee the results. We did 30 repetitions for each test with CARSKit and presented the mean and standard deviation.

4.2.2 DataWig

Biessmann et al. [2018] proposes DataWig imputation using a combination of numerical representation for categorical values, NLP using LSTM for text fields and field correlation for make the imputation. Despite it was developed to deal with high volume of data, we decided to test its performance in our scenario. Like what was done in 'Most Frequent' imputation, we used the DataWig library in correlated segment of rating matrix. The table 4.5 exemplifies the result. As CARSKit accepts non-integer numbers as input, we decided to use continuous imputation method.

UI	trafficConditions- traffic jam	sleepiness- awake	landscape- mountains
1001-715	2.00	3.55	1.77
1001-279	0.99	2.00	2.29
1001-694	1.78	4.00	2.60
1001-251	1.01	2.00	2.42
1002-762	1.00	1.70	0.89

Table 4.5: DataWig Imputation Example.

The results of DataWig imputation (fig 4.10), similar to *Most Frequent* some improvements in some correlated columns imputation. It shows more precise than *Most Frequent*, where RMSE present some improvements. For the Cumulative group imputation (fig 4.11) all the values are bigger (worse) compared than the original.

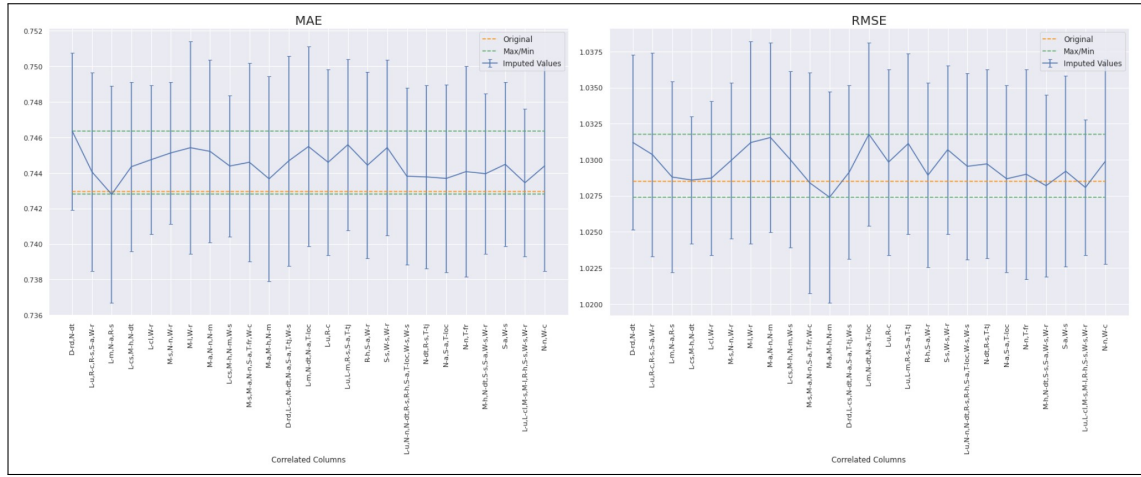


Figure 4.10: MAE and RMSE Results for each group imputation - DataWig

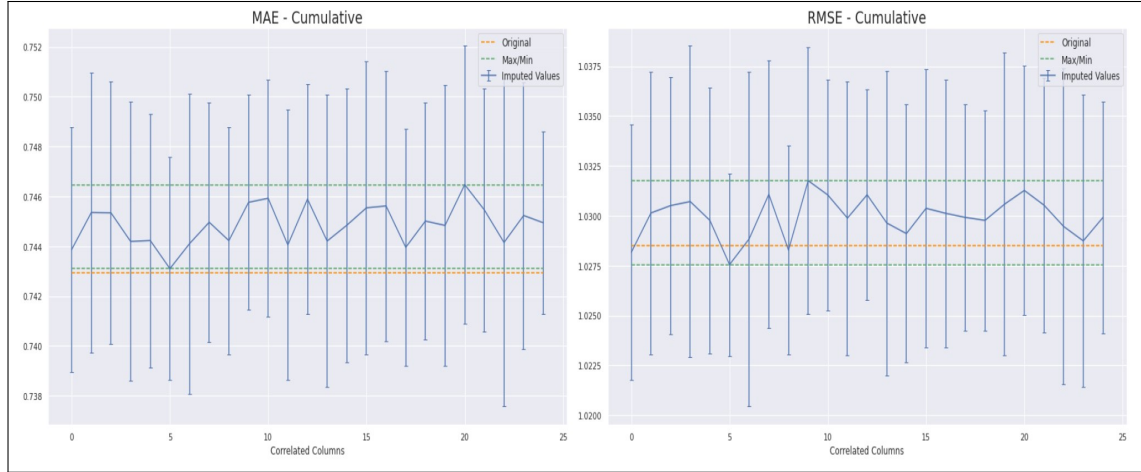


Figure 4.11: MAE and RMSE Results for Cumulative group imputation - DataWig

4.2.3 Factorization Machines

There are several works presenting good results in application of FM in sparse datasets [Rendle, 2010, Juan et al., 2016]. Considering FM shows good results in data sparsity environments, better than classical SVM, we decided use it to make imputation. Similar to DataWig, we use the influence of others columns to create a model and impute a value to '*User-Item-Context*' when exists correlated context to make the prediction.

For realize the imputation tests we used the library *pywFM*, an adaptation of *libFM* [Rendle, 2012]. To assure the reproducibility, we fixed the random seed. For verify the influence of the number of interactions, we tested its influence in the result and the computation time, presented in figure 4.12. The FM value results has few modification with more than 100 interactions, and the execution time increases exponentially. We adopted the value of 100 interactions.

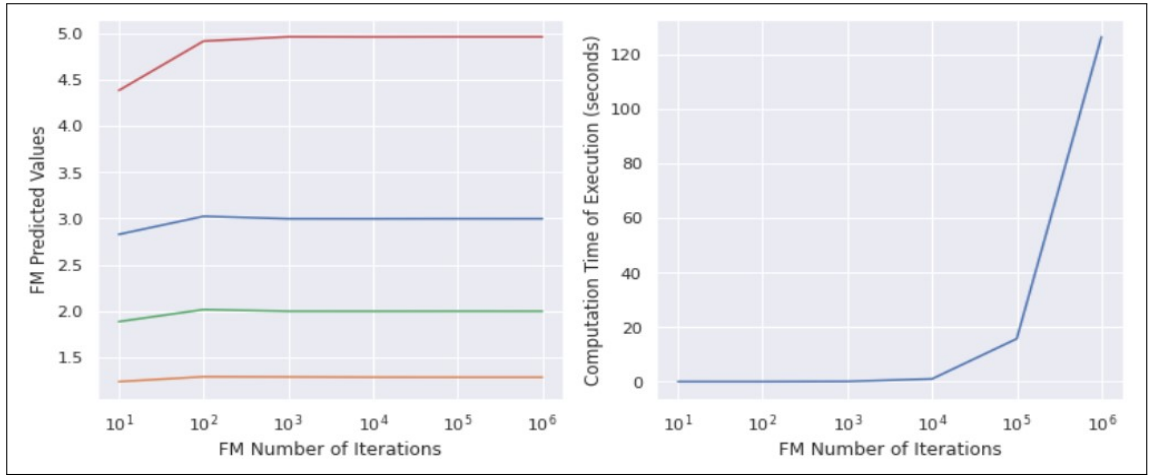


Figure 4.12: Impact of Number of Iterations in the FM result and computation time

The *libFM* allows make *regression* or *binary classification*. The regression consist in define a model that estimate the relation between the variables and presents a continous value. In other hand, the binary classification consists in just two options (e.g. 0 or 1, True or False). Like we need make the imputation between 1 and 5, regression are our option. The learning method could be Stochastic Gradient Descent (SGD), Alternating Least Squares (ALS) or Markov Chain Monte Carlo (MCMC). Like Rendle [2012] obtained better results in CARS using MCMC, we decided to use it.

The results of FM imputation (fig 4.13), similar to *Most Frequent* and *DataWig* shows some improvements in some correlated columns imputation. It shows more precise than *Most Frequent*, where RMSE present some improvements. The same is observed in cumulative group imputation (fig 4.14).

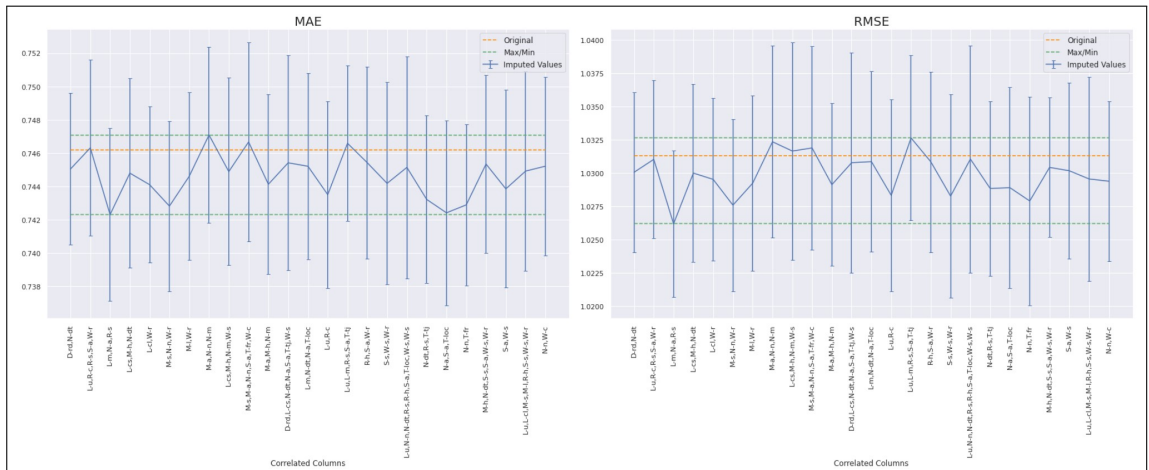


Figure 4.13: MAE and RMSE Results for each group imputation - FM

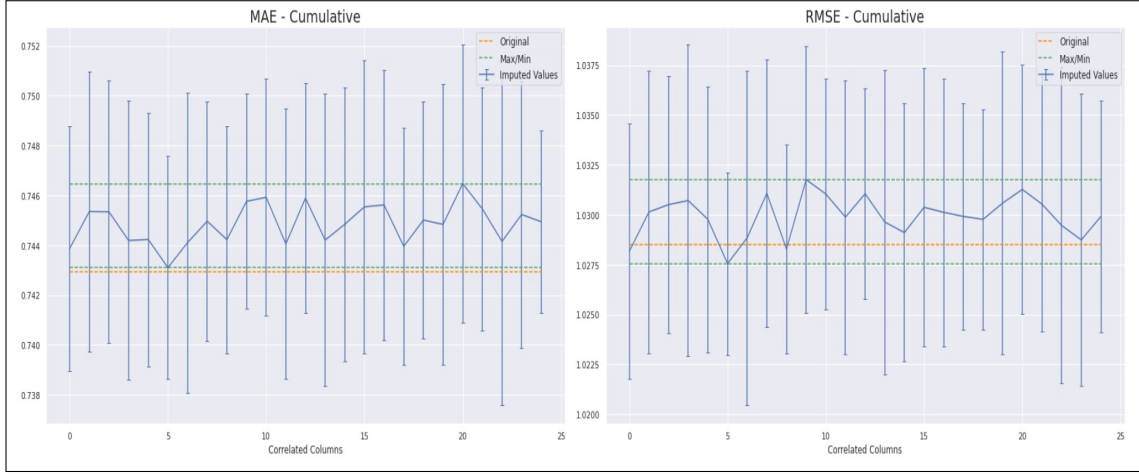


Figure 4.14: MAE and RMSE Results for Cumulative group imputation - FM

4.2.4 Discussion

The approach of using data imputation in less sparse Matrix segments present some improvements compared with the original dataset, where the MAE and RMSE got small values compared with the base line. Basically all three imputations methods have at least one case with improvements.

The FM imputation method presents the best overall results, with improvements in almost all matrix segments (fig 4.13), with 21 occurrences of improvements in MAE and RMSE over 25 correlated sub matrices. The *Most Frequent* Imputation got 16 occurrences. The DataWig method got the worst results (fig 4.10), with just 1 case of improvement.

Against expectations, the cumulative imputations of correlated segments got improvements compared the base line (figures 4.11 and 4.14), but not better compared with the individual. Using cumulative imputation, we got a smaller sparsity, but not better RS results. So, in the tested dataset, the application of imputation in all possible correlation cases, do not improve the results.

The best improvement occurs using FM imputation for an specific correlated columns, with an improvement of 0.54% in MAE and 0.48% in RMSE (correlated columns Landscape-mountain, NaturalPhenomena-afternoon, RoadType-serpentine).

Comparing the Overall Sparsity for each group imputation (fig 4.9 with the improvements of MAE and RMSE (fig 4.13), we can verify that the less sparsity do not causes better RS results.

Chapter 5

Conclusion and Future Work

In this project, we tested two methods to treat the data sparsity in CARS. The first method consists in use proximity of word vectors to group the contexts conditions and factors. The main idea is try to use the meaning of the words (conditions and factors), characterized by word vectors from pre-trained models (e.g. FastText, GloVe), to group similar context conditions to improve the RS.

We tested three different *FastText* pre-trained models, created from very distinct types of data. Comparing the results (table 3.1), the more human explainable result was obtained using the model from DBPedia because it structured characteristics. Using the weekdays example, it is more easy to consider *saturday* and *sunday* (weekends) together than *sunday* and *wednesday*. These word vector proximity depends on the pre-trained model.

Our tests do not give improvements and shows big dependency on the pre-trained model and in the how significant is the word used to describe the conditions and factors. For example, in InCarsMusic dataset the columns titles (Table 4.1) use words together, like 'naturalphenomena' and 'DrivingStyle'. In this case, if titles are not treat, the pre-trained model probably are do not have vector associated to the word. In other datasets, the factors and conditions are just acronyms or references (e.g. 'DS', 'C1', 'F2'), and without a data dictionary this method can not be applied.

The generalization using word vectors proximity, in general, do not give consistent improvements, given worst MAE and RMSE compared with the original dataset (figures 4.4 and 4.2). For future work, some tests with another CARS datasets could verify the consistence the results of this method and test different CARS algorithm different than CAMF_CU.

The second method consist in make data imputation in less sparse matrix segment. Considering that the data imputation gives better results when exist just few missing values, we tried to transform the matrix and create small segments, with just some columns and rows. Comparing segments using graphs connections and conditions correlation, the more dense segments are obtained using conditions correlations. In these less dense small matrix segment, we applied 3 imputation methods: Most Frequent, DataWig and Factorization Machines - FM. The DataWig, that is more adapted to treat word context, did not give good results. The FM gives better results than Most Frequent imputation.

To verify if we concatenate the improvements of each imputation for every matrix segment, we added the new imputed ratings. But the results of RS did not improved consistently. It indicates some factor-context correlations imputation need to be considered carefully before be used. More research is necessary to understand why

some correlations give better improvements than others.

Comparing FM and Most Frequent imputations, the best results are for FM, but not with much improvement. In another hand, the Most Frequent imputation requires less computation time. For FM, when the number of iterations of the algorithm is increased, the computation time also increases exponentially (fig 4.12). So FM need to be correctly balanced in production environments, and the Most Frequent could be an option in case of high demand. As the proposed methods requires significant matrix manipulation, in production RS, offline approach could be more feasible.

The improvements of FM are 0.5%. Despite it looks not so significant initially, applying it frequently and in big environments could result in relevant result for the quality of RS. And need to consider the test dataset is highly sparse, so the 0.5% of improvements is relevant in these conditions.

The method to segment more dense matrix depends in exist correlated contexts in the dataset and the level of correlation. In this study we used the InCarsMusic dataset and correlation coefficient bigger than 0.99. More studies are necessary to verify the impact of use different correlation coefficient threshold and the application in others CARS dataset.

Bibliography

- G. D. Abowd, A. K. Dey, P. J. Brown, N. Davies, M. Smith, and P. Steggles. Towards a better understanding of context and context-awareness. In *International symposium on handheld and ubiquitous computing*, pages 304–307. Springer, 1999.
- G. Adomavicius and A. Tuzhilin. Extending recommender systems: A multidimensional approach. In *Proceedings of the International Joint Conference on Artificial Intelligence (IJCAI-01), Workshop on Intelligent Techniques for Web Personalization (ITWP2001), Seattle, Washington, August*, pages 4–6. Citeseer, 2001.
- G. Adomavicius and A. Tuzhilin. Context-aware recommender systems. In *Recommender systems handbook*, pages 217–253. Springer, 2011.
- G. Adomavicius, R. Sankaranarayanan, S. Sen, and A. Tuzhilin. Incorporating contextual information in recommender systems using a multidimensional approach. *ACM Transactions on Information Systems (TOIS)*, 23(1):103–145, 2005.
- G. Adomavicius, N. Manouselis, and Y. Kwon. Multi-criteria recommender systems. In *Recommender systems handbook*, pages 769–803. Springer, 2011.
- C. C. Aggarwal. *Data mining: the textbook*. Springer, 2015.
- C. C. Aggarwal et al. *Recommender systems*, volume 1. Springer, 2016.
- D. Anand and K. K. Bharadwaj. Utilizing various sparsity measures for enhancing accuracy of collaborative recommender systems based on local and global similarities. *Expert systems with applications*, 38(5):5101–5109, 2011.
- S. Auer, C. Bizer, G. Kobilarov, J. Lehmann, R. Cyganiak, and Z. Ives. Dbpedia: A nucleus for a web of open data. In *The semantic web*, pages 722–735. Springer, 2007.
- M. J. Azur, E. A. Stuart, C. Frangakis, and P. J. Leaf. Multiple imputation by chained equations: what is it and how does it work? *International journal of methods in psychiatric research*, 20(1):40–49, 2011.
- L. Baltrunas, M. Kaminskas, B. Ludwig, O. Moling, F. Ricci, A. Aydin, K.-H. Lücke, and R. Schwaiger. Incarmusic: Context-aware music recommendations in a car. In *E-Commerce and Web Technologies*, pages 89–100. Springer, 2011a.
- L. Baltrunas, B. Ludwig, and F. Ricci. Matrix factorization techniques for context aware recommendation. In *Proceedings of the fifth ACM conference on Recommender systems*, pages 301–304, 2011b.
- L. Baltrunas, K. Church, A. Karatzoglou, and N. Oliver. Frappe: Understanding the usage and perception of mobile app recommendations in-the-wild. *arXiv preprint arXiv:1505.03014*, 2015.

- G. E. Batista and M. C. Monard. An analysis of four missing data treatment methods for supervised learning. *Applied artificial intelligence*, 17(5-6):519–533, 2003.
- K. Bauman and A. Tuzhilin. Discovering contextual information from user reviews for recommendation purposes. In *CBRecSys@ RecSys*, pages 2–9. Citeseer, 2014.
- R. Bell, Y. Koren, and C. Volinsky. Modeling relationships at multiple scales to improve accuracy of large recommender systems. In *Proceedings of the 13th ACM SIGKDD international conference on Knowledge discovery and data mining*, pages 95–104, 2007.
- S. Bergamaschi and L. Po. Comparing lda and lsa topic models for content-based movie recommendation systems. In *International conference on web information systems and technologies*, pages 247–263. Springer, 2014.
- F. Biessmann, D. Salinas, S. Schelter, P. Schmidt, and D. Lange. ” deep” learning for missing value imputation in tables with non-numerical data. In *Proceedings of the 27th ACM International Conference on Information and Knowledge Management*, pages 2017–2025, 2018.
- J. Bobadilla, F. Ortega, A. Hernando, and A. Gutiérrez. Recommender systems survey. *Knowledge-based systems*, 46:109–132, 2013.
- P. Bojanowski, E. Grave, A. Joulin, and T. Mikolov. Enriching word vectors with subword information. *Transactions of the Association for Computational Linguistics*, 5:135–146, 2017.
- M. Brand. Fast online svd revisions for lightweight recommender systems. In *Proceedings of the 2003 SIAM international conference on data mining*, pages 37–46. SIAM, 2003.
- M. Braunhofer, M. Elahi, and F. Ricci. Sts: A context-aware mobile recommender system for places of interest. In *Posters, Demos, Late-breaking Results and Workshop Proceedings of the 22nd Conference on User Modeling, Adaptation, and Personalization, co-located with the 22nd Conference on User Modeling, Adaptation, and Personalization*. CEUR, 2014.
- I. Cantador, I. Fernández-Tobías, S. Berkovsky, and P. Cremonesi. Cross-domain recommender systems. In *Recommender systems handbook*, pages 919–959. Springer, 2015.
- L. O. Colombo-Mendoza, R. Valencia-García, A. Rodríguez-González, G. Alor-Hernández, and J. J. Samper-Zapater. Recommetz: A context-aware knowledge-based mobile recommender system for movie showtimes. *Expert Systems with Applications*, 42(3):1202–1222, 2015.
- M. Deshpande and G. Karypis. Item-based top-n recommendation algorithms. *ACM Transactions on Information Systems (TOIS)*, 22(1):143–177, 2004.
- J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova. Bert: Pre-training of deep bidirectional transformers for language understanding. *arXiv preprint arXiv:1810.04805*, 2018.

- K. Ethayarajh. How contextual are contextualized word representations? comparing the geometry of bert, elmo, and gpt-2 embeddings. *arXiv preprint arXiv:1909.00512*, 2019.
- J. Ghosh and A. Strehl. Similarity-based text clustering: A comparative study. In *Grouping Multidimensional Data*, pages 73–97. Springer, 2006.
- W. H. Gomaa, A. A. Fahmy, et al. A survey of text similarity approaches. *International Journal of Computer Applications*, 68(13):13–18, 2013.
- J. Han, J. Pei, and M. Kamber. *Data mining: concepts and techniques*. Elsevier, 2011.
- F. E. Harrell Jr. *Regression modeling strategies: with applications to linear models, logistic and ordinal regression, and survival analysis*. Springer, 2015.
- K. Haruna, M. Akmar Ismail, S. Suhendroyono, D. Damiasih, A. C. Pierewan, H. Chiroma, and T. Herawan. Context-aware recommender system: A review of recent developmental process and future research direction. *Applied Sciences*, 7(12):1211, 2017.
- J. L. Herlocker, J. A. Konstan, L. G. Terveen, and J. T. Riedl. Evaluating collaborative filtering recommender systems. *ACM Transactions on Information Systems (TOIS)*, 22(1):5–53, 2004.
- T. Hofmann. Probabilistic latent semantic indexing. In *Proceedings of the 22nd annual international ACM SIGIR conference on Research and development in information retrieval*, pages 50–57, 1999.
- M. Hu and B. Liu. Mining and summarizing customer reviews. In *Proceedings of the tenth ACM SIGKDD international conference on Knowledge discovery and data mining*, pages 168–177, 2004.
- H. Hwangbo and Y. Kim. An empirical study on the effect of data sparsity and data overlap on cross domain collaborative filtering performance. *Expert Systems with Applications*, 89:254–265, 2017.
- S. Ilarri, R. Trillo-Lado, and R. Hermoso. Datasets for context-aware recommender systems: Current context and possible directions. In *2018 IEEE 34th International Conference on Data Engineering Workshops (ICDEW)*, pages 25–28. IEEE, 2018.
- D. Jannach, M. Zanker, A. Felfernig, and G. Friedrich. *Recommender systems: an introduction*. Cambridge University Press, 2010.
- Y. Juan, Y. Zhuang, W.-S. Chin, and C.-J. Lin. Field-aware factorization machines for ctr prediction. In *Proceedings of the 10th ACM Conference on Recommender Systems*, pages 43–50, 2016.
- D. Kim, C. Park, J. Oh, S. Lee, and H. Yu. Convolutional matrix factorization for document context-aware recommendation. In *Proceedings of the 10th ACM conference on recommender systems*, pages 233–240, 2016.
- R. Kohavi et al. A study of cross-validation and bootstrap for accuracy estimation and model selection. In *Ijcai*, volume 14, pages 1137–1145. Montreal, Canada, 1995.

- Y. Koren. Collaborative filtering with temporal dynamics. In *Proceedings of the 15th ACM SIGKDD international conference on Knowledge discovery and data mining*, pages 447–456, 2009.
- A. Košir, A. Odić, M. Kunaver, M. Tkalčič, and J. Tasič. Database for contextual personalization. *Elektrotehniski vestnik*, 2011.
- R. Krestel, P. Fankhauser, and W. Nejdl. Latent dirichlet allocation for tag recommendation. In *Proceedings of the third ACM conference on Recommender systems*, pages 61–68, 2009.
- B. Lamche, E. Pollok, W. Wörndl, and G. Groh. Evaluating the effectiveness of stereotype user models for recommendations on mobile devices. In *UMAP Workshops*. Citeseer, 2014.
- Y. LeCun, Y. Bengio, and G. Hinton. Deep learning. *nature*, 521(7553):436–444, 2015.
- S. K. Lee, Y. H. Cho, and S. H. Kim. Collaborative filtering with ordinal scale-based implicit ratings for mobile music recommendations. *Information Sciences*, 180(11):2142–2155, 2010.
- O. Maimon and L. Rokach. Decomposition methodology for knowledge discovery and data mining. In *Data mining and knowledge discovery handbook*, pages 981–1003. Springer, 2005.
- O. S. Miettinen. *Theoretical epidemiology: principles of occurrence research in medicine*. Wiley New York, 1985.
- T. Mikolov, I. Sutskever, K. Chen, G. S. Corrado, and J. Dean. Distributed representations of words and phrases and their compositionality. In *Advances in neural information processing systems*, pages 3111–3119, 2013.
- W. I. D. Mining. Data mining: Concepts and techniques. *Morgan Kaufmann*, 10: 559–569, 2006.
- C. Musto, G. Semeraro, M. De Gemmis, and P. Lops. Word embedding techniques for content-based recommender systems: An empirical evaluation. In *Recsys posters*, 2015.
- M. A. Nielsen. *Neural networks and deep learning*, volume 2018. Determination press San Francisco, CA, 2015.
- M. Nilashi, O. bin Ibrahim, N. Ithnin, and N. H. Sarmin. A multi-criteria collaborative filtering recommender system for the tourism domain using expectation maximization (em) and pca-anfis. *Electronic Commerce Research and Applications*, 14(6):542–562, 2015.
- J. Niu, L. Wang, X. Liu, and S. Yu. Fuir: Fusing user and item information to deal with data sparsity by using side information in recommendation systems. *Journal of Network and Computer Applications*, 70:41–50, 2016.
- C. Obeid, I. Lahoud, H. El Khoury, and P.-A. Champin. Ontology-based recommender system in higher education. In *Companion Proceedings of the The Web Conference 2018*, pages 1031–1034, 2018.

- M. G. Ozsoy. From word embeddings to item recommendation. *CoRR*, abs/1601.01356:1, 2016. URL <http://arxiv.org/abs/1601.01356>.
- J. Pennington, R. Socher, and C. D. Manning. Glove: Global vectors for word representation. In *Proceedings of the 2014 conference on empirical methods in natural language processing (EMNLP)*, pages 1532–1543, 2014.
- T. M. Phuong, N. D. Phuong, et al. Graph-based context-aware collaborative filtering. *Expert Systems with Applications*, 126:9–19, 2019.
- V. Powell and L. Lehe. Principal component analysis. URL <http://setosa.io/ev/principalcomponent-analysis>, 1(1):1, 2015.
- P. Pu, L. Chen, and R. Hu. A user-centric evaluation framework for recommender systems. In *Proceedings of the fifth ACM conference on Recommender systems*, pages 157–164, 2011.
- S. Punjabi and P. Bhatt. Robust factorization machines for user response prediction. In *Proceedings of the 2018 World Wide Web Conference*, pages 669–678, 2018.
- J. Ramos et al. Using tf-idf to determine word relevance in document queries. In *Proceedings of the first instructional conference on machine learning*, volume 242, pages 133–142. New Jersey, USA, 2003.
- S. Rendle. Factorization machines. In *2010 IEEE International Conference on Data Mining*, pages 995–1000. IEEE, 2010.
- S. Rendle. Factorization machines with libfm. *ACM Transactions on Intelligent Systems and Technology (TIST)*, 3(3):1–22, 2012.
- F. Ricci, L. Rokach, and B. Shapira. Introduction to recommender systems handbook. In *Recommender systems handbook*, pages 1–35. Springer, 2011.
- F. Ricci, L. Rokach, and B. Shapira. Recommender systems: introduction and challenges. In *Recommender systems handbook*, pages 1–34. Springer, 2015.
- T. Rukat, D. Lange, S. Schelter, and F. Biessmann. Towards automated data quality management for machine learning. *Amazon Research*, 2019.
- A. I. Schein, A. Popescul, L. H. Ungar, and D. M. Pennock. Methods and metrics for cold-start recommendations. In *Proceedings of the 25th annual international ACM SIGIR conference on Research and development in information retrieval*, pages 253–260, 2002.
- C. Sidige, S. Pothula, R. Korra, M. N. Kumar, and M. Kumar. Normalized distance measure: A measure for evaluating mlir merging mechanisms. *International Journal of Computer Science Issues (IJCSI)*, 8(3):209, 2011.
- T. Silveira, M. Zhang, X. Lin, Y. Liu, and S. Ma. How good your recommender system is? a survey on evaluations in recommendation. *International Journal of Machine Learning and Cybernetics*, 10(5):813–831, 2019.
- N. Stadler and P. Buhlmann. Missing values: sparse inverse covariance estimation and an extension to sparse regression. *Statistics and Computing*, 22(1):219–235, 2012.

- A. Strehl, J. Ghosh, and R. Mooney. Impact of similarity measures on web-page clustering. In *Workshop on artificial intelligence for web search (AAAI 2000)*, volume 58, page 64, 2000.
- X. Su and T. M. Khoshgoftaar. A survey of collaborative filtering techniques. *Advances in artificial intelligence*, 2009:1687–7470, 2009.
- A. Umyarov and A. Tuzhilin. Leveraging aggregate ratings for better recommendations. In *Proceedings of the 2007 ACM conference on Recommender systems*, pages 161–164, 2007.
- Z. Vahidi Ferdousi, E. Negre, and D. Colazzo. Context factors in context-aware recommender systems. In *AISR 2017 : Atelier interdisciplinaire sur les systèmes de recommandation*, Paris, France, May 2017.
- S. Van Buuren. *Flexible imputation of missing data*. CRC press, 2018.
- M. G. Vozalis and K. G. Margaritis. A recommender system using principal component analysis. In *Published in 11th panhellenic conference in informatics*, pages 271–283, 2007.
- Y. Zheng, R. Burke, and B. Mobasher. Differential context relaxation for context-aware travel recommendation. In *International Conference on Electronic Commerce and Web Technologies*, pages 88–99. Springer, 2012.
- Y. Zheng, R. Burke, and B. Mobasher. Splitting approaches for context-aware recommendation: An empirical study. In *Proceedings of the 29th Annual ACM Symposium on Applied Computing*, pages 274–279, 2014.
- Y. Zheng, B. Mobasher, and R. Burke. Carskit: A java-based context-aware recommendation engine. In *2015 IEEE International Conference on Data Mining Workshop (ICDMW)*, pages 1668–1671. IEEE, 2015.
- Y. Zhou. Recommender systems for contextually-aware, versioned items. In *Proceedings of the 13th ACM Conference on Recommender Systems*, pages 606–610, 2019.
- D. Zou, M. Sheng, H. Yu, J. Mao, S. Chen, and W. Sheng. Factorized weight interaction neural networks for sparse feature prediction. *Neural Computing and Applications*, pages 1–13, 2019.